

UNIVERSIDAD AUTÓNOMA METROPOLITANA

Unidad Azcapotzalco

División de Ciencias Básicas e Ingeniería
Licenciatura en Ingeniería en Computación

**Intervalos de confianza de la evaluación del polinomio cromático de una
gráfica utilizando el método de Montecarlo**

Proyecto de Investigación

Trimestre 2025-I

Proyecto UAM “Resolución de problemas de gráficas usando el polinomio
cromático”

Clave CB003-23

Luis Daniel Ramos López

2193039862

Alumno

Dra. Ma. Guadalupe Rodríguez

Sánchez

Asesora

Dr. Rodrigo Alexander Castro

Campos

Co-Asesor

Fecha 16/01/2025

Declaratorias

Yo, María Guadalupe Rodríguez Sánchez, declaro que aprobé el contenido del presente Reporte de Proyecto de Integración y doy mi autorización para su publicación en la Biblioteca Digital, así como en el Repositorio Institucional de UAM Azcapotzalco.

A handwritten signature in blue ink, consisting of a large, stylized 'M' and 'R' intertwined, with a vertical line through the center.

Dra. María Guadalupe Rodríguez Sánchez

Yo, Rodrigo Alexander Castro Campos, declaro que aprobé el contenido del presente Reporte de Proyecto de Integración y doy mi autorización para su publicación en la Biblioteca Digital, así como en el Repositorio Institucional de UAM Azcapotzalco.

A handwritten signature in blue ink, starting with a large 'R' and 'C' and ending with a flourish.

Dr. Rodrigo Alexander Castro Campos

Yo, Luis Daniel Ramos López, doy mi autorización a la Coordinación de Servicios de Información de la Universidad Autónoma Metropolitana, Unidad Azcapotzalco, para publicar el presente documento en la Biblioteca Digital, así como en el Repositorio Institucional de UAM Azcapotzalco.

A handwritten signature in blue ink, featuring a large 'L' and 'R' with a horizontal line through the middle.

Luis Daniel Ramos López

Resumen

El presente proyecto se enfoca en la implementación del método de Montecarlo para la evaluación aproximada del polinomio cromático, el cual es una herramienta fundamental en la teoría de gráficas y problemas combinatorios. Dado que el cálculo exacto de este polinomio es un problema #P-completo, se emplea una estrategia probabilística basada en la simulación aleatoria de coloraciones válidas para estimar una evaluación para algún valor de su variable asociada.

Tabla de contenido

1. Introducción	6
2. Antecedentes	6
3. Justificación.....	7
4. Objetivos	8
4.1. Objetivo general	8
4.2. Objetivos específicos	8
5. Marco Teórico	8
5.1. Conceptos de gráficas	8
5.1.1. Definiciones fundamentales de gráficas.....	8
5.1.2. Coloración de gráficas.....	9
5.1.3. Polinomio cromático de una gráfica	9
5.2. Métodos probabilísticos	10
5.2.1. Conceptos básicos de probabilidad	10
5.2.2. El método de Montecarlo	11
6. Desarrollo del proyecto	11
6.1. Desarrollo de los algoritmos a partir del marco teórico	11
6.1.1. Método de Montecarlo para calcular $P(G, k)$ para una k dada.....	11
6.1.2. Intervalos de confianza de $P(G, k)$ para una k dada.....	14
6.2. Módulo de coloraciones aleatorias.....	15
6.3. Módulo de evaluación estadística	16
6.4. Módulo de operaciones de números enteros y flotantes de 128 bits.....	16
6.5. Módulo adaptativo de simulaciones.....	16
7. Resultados	17
7.1. Método de Montecarlo vs borrado-contracción.	18
7.2. Método de Montecarlo vs evaluaciones de polinomios conocidos.....	19
7.2.1. Método de Montecarlo para gráficas completas	19
7.2.2. Método de Montecarlo para gráficas sin aristas.....	20
7.2.3. Método de Montecarlo para caminos.....	20
7.2.4. Peor de los casos del método de Montecarlo	22
8. Análisis de los resultados	22
8.1. Cálculo del error garantizado e_G para el intervalo de confianza.....	22
8.2. Análisis del método de Montecarlo vs borrado-contracción.	22
8.2. Análisis del método de Montecarlo vs evaluaciones de polinomios conocidos.....	23
8.2.2. Análisis del método de Montecarlo para gráficas completas	23
8.2.3. Análisis del método de Montecarlo para gráficas sin aristas	24
8.2.4. Análisis del método de Montecarlo para caminos.....	25
8.2.5. Análisis del peor de los casos del método de Montecarlo.....	25
8.3. Conclusión de análisis de resultados.....	26
9. Conclusiones	26

10. Referencias Bibliográficas	27
12. Entregables	28
12.1. Pseudocódigo de los algoritmos.....	28
12.2. Código fuente del módulo de generación de coloraciones en C++	29
12.3. Código fuente del módulo de evaluación estadística en C++	30
12.4. Código fuente del módulo de operaciones de 128 bits en C++	30
12.5. Código fuente del módulo adaptativo de simulaciones en C++.....	31
12.6. Código fuente del módulo del algoritmo de Borrado-Contracción en C++.....	33
12.7. Códigos fuentes implementados para los resultados de la sección 7	35

1. Introducción

La teoría de la complejidad computacional estudia problemas cuya resolución requiere recursos significativos, ya sea en tiempo o espacio. En este contexto, los problemas $\#P$ se destacan por abordar el conteo de soluciones de problemas de decisión de la clase NP , apareciendo en diversas disciplinas como la teoría de gráficas, la optimización y la física estadística. Dentro de esta clase, los problemas $\#P$ -completos son especialmente relevantes, ya que representan algunos de los retos más complejos en el ámbito del conteo.

Un problema $\#P$ -completo particularmente importante es el cálculo del polinomio cromático $P(G, k)$, el cual determina el número de formas válidas de colorear los vértices de una gráfica G con k colores, de manera que vértices adyacentes no compartan el mismo color. Estas coloraciones, conocidas como coloraciones propias, no solo son fundamentales en la teoría de gráficas, sino que también encuentran aplicaciones en áreas como la asignación de recursos, el diseño de redes y el modelado de sistemas físicos. Sin embargo, calcular $P(G, k)$ es NP -Duro, lo que lo convierte en un problema intratable para gráficas grandes y limita su aplicación práctica en contextos complejos.

Ante estas dificultades, los métodos probabilísticos como el de Montecarlo ofrecen una alternativa viable para aproximar el valor del polinomio cromático. Este enfoque se basa en el muestreo aleatorio para explorar el espacio de soluciones, proporcionando estimaciones eficientes y escalables en casos donde los métodos exactos son impracticables. Si bien no garantiza valores exactos, el método de Montecarlo permite obtener estimaciones precisas con control del error estándar y generación de intervalos de confianza, lo que lo convierte en una herramienta poderosa para estudiar gráficas de gran tamaño y complejidad.

Este trabajo presenta una implementación del método de Montecarlo para la estimación del polinomio cromático en gráficas. Se desarrollaron tres módulos principales: generación de coloraciones aleatorias, evaluación estadística y simulaciones adaptativas. Además, se analizaron las relaciones entre el tamaño de la muestra, el error relativo y la estabilidad de las estimaciones, ajustando dinámicamente el número de simulaciones para cumplir con niveles de confianza predefinidos. Finalmente, se evaluó la eficiencia del método en gráficas de diferente complejidad, comparando los resultados obtenidos con valores exactos conocidos para validar la precisión del enfoque. Este reporte documenta el desarrollo, análisis y resultados obtenidos, proporcionando una base sólida para futuras aplicaciones de este método en problemas combinatorios similares.

2. Antecedentes

El cálculo del polinomio cromático, un problema $\#P$ -completo, se encuentra dentro de los problemas más complejos en la teoría de la computación, ya que implica contar soluciones en espacios combinatorios exponencialmente grandes. Aunque los métodos exactos son efectivos en casos pequeños, su intratabilidad en gráficas grandes ha llevado a la exploración de enfoques probabilísticos como el método de Montecarlo. Este método ha demostrado ser una herramienta poderosa en diversos problemas de conteo combinatorio, ofreciendo aproximaciones precisas y eficientes en escenarios donde los métodos deterministas fallan.

Diversos estudios han explorado la efectividad de Montecarlo para abordar problemas combinatorios relacionados con conteo. Por ejemplo, en el caso del problema clásico de las N -reinas, se han implementado técnicas como muestreo de importancia, recocido simulado y métodos de reordenamiento, logrando estimaciones rápidas y confiables incluso para tableros grandes. Los autores han destacado cómo técnicas avanzadas, como el algoritmo Swendsen-Wang para matrices

binarias, optimizan el muestreo y mejoran la exploración del espacio de soluciones, reduciendo significativamente la cantidad de simulaciones necesarias para alcanzar una precisión deseada [1].

En otro ejemplo, al abordar problemas más estructurados como los cuadrados latinos, los investigadores han transformado estos desafíos en sistemas termodinámicos, permitiendo aplicar Montecarlo para calcular funciones de partición y estimar soluciones. Movimientos colectivos, como intercambios en configuraciones y algoritmos de clúster, han mostrado ser particularmente efectivos para explorar regiones densas del espacio de soluciones, demostrando que Montecarlo puede adaptarse a la naturaleza específica de cada problema [2].

El problema de las extensiones lineales en conjuntos parcialmente ordenados, que también pertenece a la clase $\#P$ -completos, ha sido abordado mediante muestreo secuencial de importancia. Este enfoque permite asignar pesos a las muestras generadas, corrigiendo los sesgos del proceso y garantizando estimaciones precisas incluso cuando el espacio de soluciones es altamente irregular. La capacidad de Montecarlo para manejar esta variabilidad destaca su flexibilidad y robustez [3].

En términos más generales, se ha demostrado que Montecarlo es aplicable incluso en problemas abstractos como la satisfactibilidad booleana (SAT) o el conteo de soluciones en fórmulas DNF. En estos casos, el método se ha utilizado para diseñar algoritmos con complejidad temporal polinómica y errores relativos acotados, lo que refuerza su viabilidad como herramienta para problemas combinatorios intratables [4]. Los resultados obtenidos en estos estudios resaltan la capacidad de Montecarlo para adaptarse a distintos contextos combinatorios, proporcionando estimaciones confiables en escenarios donde otros métodos fallan.

Los trabajos mencionados no solo validan la eficacia del método de Montecarlo en una variedad de problemas combinatorios, sino que también muestran cómo las técnicas de optimización específicas, como muestreo adaptativo y control estadístico, mejoran aún más la precisión de las estimaciones. Estas estrategias son directamente aplicables al cálculo aproximado del polinomio cromático, donde el tamaño del espacio de soluciones y la complejidad combinatoria representan desafíos similares. Así, el presente proyecto se fundamenta en una metodología comprobada, aprovechando las fortalezas de Montecarlo para abordar problemas intratables de manera eficiente y escalable.

3. Justificación

La implementación del polinomio cromático como eje central de este proyecto es de gran relevancia tanto en el ámbito teórico como en el práctico, especialmente considerando su complejidad intrínseca al ser un problema $\#P$ -completo. El cálculo del polinomio cromático, $P(G, k)$, está profundamente relacionado con problemas fundamentales en la teoría de gráficas y otros campos de la ciencia y la tecnología. Sin embargo, su naturaleza computacionalmente intratable para gráficas grandes, debido al crecimiento exponencial del espacio de soluciones, ha limitado su uso práctico, haciendo necesario el desarrollo de métodos alternativos para su aproximación.

En términos prácticos, el polinomio cromático tiene aplicaciones directas en áreas como la asignación de recursos en redes de telecomunicaciones, la programación de tareas, y el modelado de sistemas físicos. Por ejemplo, en redes inalámbricas, la asignación de frecuencias para evitar interferencias puede modelarse mediante coloraciones propias de gráficas. En física estadística, el polinomio cromático está relacionado con el modelo de Potts, utilizado para analizar sistemas de partículas en equilibrio termodinámico. Estas aplicaciones subrayan la importancia de contar con herramientas efectivas para aproximar $P(G, k)$, particularmente en contextos donde el tamaño y la complejidad de las gráficas hacen inviables los métodos exactos.

El método de Montecarlo representa una solución viable para superar estas limitaciones. Este enfoque probabilístico permite obtener estimaciones confiables del polinomio cromático sin necesidad de realizar cálculos exhaustivos, lo que lo convierte en una alternativa eficiente para gráficas de gran tamaño. Entre sus principales ventajas destacan su escalabilidad y su capacidad de ajustar dinámicamente el número de simulaciones, equilibrando precisión y costo computacional. Además, Montecarlo permite generar intervalos de confianza, ofreciendo una métrica clara para cuantificar la incertidumbre asociada a las estimaciones, lo cual es crucial en problemas de alta complejidad como este.

El presente proyecto no solo busca abordar el cálculo aproximado del polinomio cromático utilizando Montecarlo, sino que también tiene un propósito metodológico al combinar técnicas estadísticas con simulación aleatoria. Esta integración permite un control riguroso del error y asegura la estabilidad de los resultados, incluso en instancias donde los espacios de solución son extremadamente grandes. Además, la metodología desarrollada en este trabajo podría aplicarse a otros problemas combinatorios, expandiendo las posibilidades de la computación probabilística.

La justificación de este proyecto radica en su potencial para contribuir al avance en la teoría de gráficas, al proporcionar un marco metodológico que haga accesible el estudio de problemas $\#P$ -completos desde una perspectiva práctica y probabilística. Esto no solo facilitará el análisis de gráficas complejas, sino que también abrirá oportunidades para aplicaciones interdisciplinarias en áreas como telecomunicaciones, optimización, y física estadística, consolidando el valor teórico y práctico del trabajo desarrollado.

4. Objetivos

4.1. Objetivo general

Implementar un algoritmo probabilístico basado en Montecarlo para la evaluación aproximada del polinomio cromático en gráficas.

4.2. Objetivos específicos

1. Implementar el método de Montecarlo para la estimación del polinomio cromático de una gráfica, mediante el muestreo de coloraciones aleatorias de la misma.
2. Desarrollar un algoritmo de ajuste dinámico del número de simulaciones en función del error relativo deseado y la estabilidad de las estimaciones.
3. Evaluar la eficiencia y precisión del método en gráficas de diferentes tamaños y estructuras, así como con diferentes parámetros estadísticos, comparando los resultados obtenidos con valores exactos conocidos

5. Marco Teórico

5.1. Conceptos de gráficas

5.1.1. Definiciones fundamentales de gráficas

Las definiciones de esta subsección fueron tomadas de Chacón [5].

Una gráfica o grafo G es un par $G = (V, E)$, donde V es un conjunto finito (vértices) y E es un multiconjunto de pares no ordenados de vértices (aristas), denotados por $\{x, y\} | x, y \in V$. Denotamos por V al conjunto de vértices de la gráfica G y por E al conjunto de aristas de la gráfica G . Dos vértices u, v de una gráfica $G = (V, E)$ se dicen adyacentes si $\{u, v\} \in E$.

Sea $G = (V, E)$ una gráfica con v vértices y e aristas, la matriz de adyacencia de G es una matriz $A(G)[a_{ij}]_{v \times v}$, en donde a_{ij} es el número de aristas que unen v_i y v_j . Una gráfica puede representarse mediante un dibujo, así los vértices se dibujan como puntos y dos vértices que son adyacentes se unen por una línea.

5.1.2. Coloración de gráficas

Sea $G = (V, E)$ una gráfica y un conjunto de colores $C = \{a, b, \dots\}$. Una coloración de G con colores de C es una correspondencia de V en C tal que:

$$\gamma: V(G) \rightarrow C$$

Dadas $G = (V, E)$ una gráfica y C un conjunto de colores, $C = \{a, b, \dots\}$. Una coloración propia de G con colores en C , es una coloración $\gamma_P: V \rightarrow C$ tal que si $\{u, v\} \in E$, entonces $\gamma_P(u) \neq \gamma_P(v)$. Se dice que G es k -*coloreable* si existe una coloración γ_P propia de G con k colores [6]. El número cromático $\chi(G)$, de una gráfica G , es el número mínimo de colores necesario para dar una coloración propia a G .

5.1.3. Polinomio cromático de una gráfica

Sea una gráfica G . El polinomio cromático de G es una función polinómica $P(G, \lambda)$ tal que, si λ es un número entero positivo, entonces $P(G, k)$ proporciona el número de k -*coloraciones* distintas de G .

Sea $G = (V, E)$ una gráfica, y sea $e = \{v_1, v_2\}$ una arista de E , la operación $G \setminus e$ consiste en eliminar la arista e del conjunto E , mientras que la operación $G \cdot e$ es quitar tanto la arista e de E , como los vértices v_1 y v_2 de V para crear un nuevo vértice v_{12} en V , el cual tiene aristas hacia todos los vértices adyacentes a v_1 y v_2 .

Sea $G = (V, E)$ una gráfica y $e \in E$ una arista de G . Entonces [7]:

$$P(G, \lambda) = P(G \setminus e) - P(G \cdot e)$$

Sea $P(G, \lambda)$ el polinomio cromático de la gráfica G en función de λ , la función recursiva $P(\text{Gráfica } G)$ para determinar el polinomio es [8]:

```

P(Gráfica G)
  sí G no tiene aristas
    retorna  $\lambda^{|V|}$ 
  sino
    Arista e ← Una arista de E(G)
    retorna  $P(G \setminus e) - P(G \cdot e)$ 
fin

```

Sea $P(G)$ el algoritmo de borrado – contracción, aplicado en una gráfica G , la complejidad del algoritmo es:

$$O(P(G)) = O(\phi^{|V|+|E|}),$$

en donde $\phi = \frac{1+\sqrt{5}}{2}$, también llamado el número áureo [9].

5.2. Métodos probabilísticos

5.2.1. Conceptos básicos de probabilidad

Las definiciones de esta subsección son tomadas de [10].

Una población es un conjunto de elementos homogéneos en los que se desea investigar la ocurrencia de una característica o propiedad. Una variable aleatoria X es una función medible $X: \Omega \rightarrow \mathcal{E}$, desde un espacio muestral Ω , como un conjunto de resultados posibles, hasta un espacio medible E . Sean $X_1, X_2, \dots, X_N$, el conjunto de N variables aleatorias, el promedio se define como:

$$\bar{X} = \frac{1}{N} \sum_{i=1}^n X_i$$

Además, su desviación estándar se define como:

$$\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (X_i - \bar{X})^2}$$

Sea una muestra estadísticamente independiente de N observaciones $x_1, \dots, x_n$, que se toma de una población estadística con una desviación estándar de σ . El valor medio $\bar{x}$ calculado a partir de la muestra, tendrá asociado un error estándar sobre la media e_σ , dado por:

$$e_\sigma = \frac{\sigma}{\sqrt{N}}$$

El error estándar de una proporción es una estadística que indica en qué medida es probable que una proporción de muestra particular difiera de la proporción en la proporción de la población. Sea p una proporción observada en una muestra de N observaciones estadísticamente independientes, el error estándar e_p está dado por la siguiente fórmula:

$$e_p = \sqrt{\frac{p \cdot (1 - p)}{N}}$$

Sean $X_1, X_2, \dots, X_n$ variables aleatorias, independientes e idénticamente distribuidas, que tienen el mismo valor esperado μ y varianza σ^2 , entonces el promedio converge en probabilidad a μ , para cualquier número positivo ε tiene:

$$\lim_{n \rightarrow \infty} P(|\bar{X}_n - \mu| < \varepsilon) = 1$$

El nivel de confianza es una medida de la fiabilidad de una estimación en términos probabilísticos. Nos dice qué tan seguro puedes estar de que una estimación está dentro de un rango específico del valor real. El valor de Z , también conocido como el estadístico Z o puntuación

Z , es una medida que indica cuántas desviaciones estándar un valor observado se encuentra por encima o por debajo de la media de una distribución normal.

En el contexto de intervalos de confianza, el valor de Z proviene de la distribución normal estándar (media $\mu = 0$, desviación estándar $\sigma = 1$) y se utiliza para determinar el rango en el que es probable que se encuentre una estimación respecto al valor real. Este rango está directamente relacionado con el nivel de confianza deseado.

Niveles de confianza comunes y sus valores de Z :

- Nivel de confianza del 90%: $Z = 1.645$
- Nivel de confianza del 95%: $Z = 1.96$
- Nivel de confianza del 99%: $Z = 2.576$

Estos valores de Z se obtienen de tablas de la distribución normal estándar, que muestran las áreas bajo la curva correspondientes a diferentes niveles de confianza.

Se considera un buen tamaño de una muestra n , dado un nivel de confianza Z , el error relativo deseado e_R , la proporción p de individuos que poseen una característica de una población infinita, se puede calcular como:

$$n = \frac{Z^2 \cdot p \cdot (1 - p)}{e_R^2}$$

5.2.2. El método de Montecarlo

Sea una muestra estadísticamente independiente de n observaciones $x_1, \dots, x_n$ y $f(x_i)$ el valor de una función que estamos evaluando en la muestra x_i , una estimación del valor esperado μ , mediante el método de Montecarlo, es:

$$\mu \approx \frac{1}{n} \sum_{i=1}^N f(x_i)$$

El método de Montecarlo se basa en que las observaciones de la muestra sean completamente aleatorias y sin ningún sesgo [11].

Dada un nivel de confianza $\alpha\%$ asociado con una Z , un valor esperado μ calculado a partir de una muestra de n observaciones y el error e_σ de la desviación estándar de las muestras, se puede estimar un intervalo de confianza del $\alpha\%$ dado el siguiente intervalo [12]:

$$[\mu - Z \cdot e_\sigma, \mu + Z \cdot e_\sigma]$$

6. Desarrollo del proyecto

6.1. Desarrollo de los algoritmos a partir del marco teórico

6.1.1. Método de Montecarlo para calcular $P(G, k)$ para una k dada

Dada una gráfica $G = (V, E)$, una coloración γ y una función $g(G, \gamma)$; para determinar si γ es una coloración propia, se propone el siguiente algoritmo para la función g :

```

g(Gráfica G, Coloración  $\gamma$ )
  sí  $\gamma$  es coloración propia de G
    regresa 1
  sino
    regresa 0
fin

```

La proporción μ_γ representa la razón del número de coloraciones propias con respecto al total de coloraciones posibles, esta proporción está dada por:

$$\mu_\gamma = \frac{1}{k^{|V|}} \sum_{i=1}^N g(G, \gamma_i)$$

Dado que la función $g(G, \gamma)$ regresa 1 solo cuando γ es una coloración propia, podríamos reinterpretar la fórmula anterior de la siguiente manera:

$$\mu_\gamma = \frac{\text{número de coloraciones propias}}{\text{número de posibles coloraciones}}$$

El polinomio cromático cuenta cuántas de las $k^{|V|}$ coloraciones posibles son propias. Así, la proporción μ_γ cuenta qué fracción del total de coloraciones posibles son propias. Estimar el valor real de $P(G, k)$, dada la proporción μ_γ , se define como:

$$P(G, k) = \mu_\gamma \cdot k^{|V|}$$

esto es equivalente a decir que el número de coloraciones propias con k colores (el valor del polinomio cromático evaluado en k) es una fracción del total de coloraciones posibles.

En el contexto del polinomio cromático, dada una gráfica $G = (V, E)$, un conjunto de k colores, una muestra estadísticamente independiente de n coloraciones $\gamma_1 \dots, \gamma_n$ y la función $g(G, \gamma)$, una estimación de μ_γ a partir del método de Montecarlo está dada por:

$$\mu_\gamma \approx \frac{1}{n} \sum_{i=1}^n f(G, \gamma_i)$$

Además, en términos del polinomio cromático, dado que el $k^{|V|}$ crece exponencialmente con respecto a $|V|$ y polinomialmente con respecto a k , el tamaño de la muestra n la calcularemos con la fórmula del tamaño de muestra de una población infinita que, sustituyendo datos con los del polinomio, está dada por:

$$n = \frac{Z^2 \cdot \mu_\gamma \cdot (1 - \mu_\gamma)}{e_R^2}$$

Como no conocemos el valor de μ_γ , primero suponemos que $\mu_\gamma = 0.5$, para después recalcularlo iterativamente hasta que tenga un error estándar menor a e_R que estará determinado dado por:

$$e_\mu = \sqrt{\frac{\mu_\gamma \cdot (1 - \mu_\gamma)}{n}}$$

En el caso de que en alguna iteración nos encontremos con $\mu_\gamma = 0$, el método Montecarlo enfrentaría una dificultad para generar una estimación representativa del polinomio cromático, ya que cuando sucede este caso entonces $e_\mu = 0$, haciendo que el método pare y nos devuelva un erróneo $P(G, k) = 0$. Para abordar este problema, se introduce un ajuste dinámico del tamaño de la muestra.

Si $\mu_\gamma = 0$, el número n de muestras se incrementará multiplicándolo por un factor f , por lo tanto:

$$n = f \cdot \frac{Z^2 \cdot \mu_\gamma \cdot (1 - \mu_\gamma)}{e_R^2}$$

donde el factor inicialmente tendrá el valor de $f = 1$ y se recalculará de la siguiente manera:

$$f \leftarrow 2 \cdot f$$

cada vez que en una iteración se tenga $\mu_\gamma = 0$.

Además, para gráficas muy grandes, dado que e_μ depende del número n de muestras utilizadas, cuando n es pequeña y tenemos un espacio de soluciones muy grande, el término e_μ no suele converger hacia el error relativo deseado ($e_\mu \leq e_R$). Para resolver esta situación, cuando el error relativo e_μ es mayor que el error relativo deseado e_R , se requiere un aumento proporcional en el número de muestras para reducir e_μ que se obtendrá en la siguiente iteración recalculando el factor f como sigue:

$$f \leftarrow f \cdot \left(1 + \frac{e_\mu - e_R}{e_R}\right)$$

Este ajuste asegura que el crecimiento de n sea mayor cuando el exceso de error $e_\mu - e_R$ es significativo, pero más conservador cuando e_μ está cerca del valor deseado para evitar aumentar el espacio de muestras del algoritmo de manera innecesaria.

Por último, para evitar que el número de simulaciones n crezca indeterminadamente, cuando n sea mayor que $k^{|V|}$ y la iteración nos dio que $e_\mu = 0$ quiere decir que es prácticamente un hecho que la evaluación del polinomio es cero.

Por lo tanto, el algoritmo para calcular una evaluación del polinomio cromático $P(G, k)$ de una gráfica $G = (V, E)$, con k colores, un error relativo e_R y un nivel de confianza Z utilizando el método de Montecarlo será el siguiente:

Montecarlo(Grafica G , Colores k , Error rel. e_R , Nivel de confianza Z)

```

    flotante  $f \leftarrow 1$ ,  $e_\mu \leftarrow 1$ ,  $\mu_\gamma \leftarrow 0.5$ 
    hacer
        entero  $n \leftarrow (f \cdot Z^2 \cdot \mu_\gamma \cdot (1 - \mu_\gamma)) / e_R^2$ 
        entero  $val \leftarrow 0$ 
        desde ( $i \leftarrow 0$ ) hasta  $n$ 
            hacer
                Coloración  $\gamma \leftarrow \text{coloraciónAleatoria}(G, k)$ 
                 $val \leftarrow val + g(G, \gamma)$ 
                 $i \leftarrow i + 1$ 
        fin
        flotante  $\mu_{ant} \leftarrow val / n$ 
        si ( $\mu_{act} = 0$ )
            hacer
                sí ( $n > k^{|V|}$ ) entonces
                    regresa 0
                fin sino
                     $f \leftarrow 2 \cdot f$ 
                    continuar
            fin
        fin
         $\mu_\gamma \leftarrow \mu_{act}$ 
         $e_\mu \leftarrow \sqrt{\mu_\gamma \cdot (1 - \mu_\gamma) / n}$ 
        si ( $e_\mu > e_R$ )
            hacer
                 $f \leftarrow f \cdot ((1 + (e_\mu - e_R)) / e_R)$ 
            fin
        fin
    mientras ( $e_\mu > e_R$ )
        regresa  $\mu_\gamma \cdot k^{|V|}$ 
fin

```

6.1.2. Intervalos de confianza de $P(G, k)$ para una k dada

Debido a que el espacio de nuestra función $f(G, c_i)$ es discreto, en vez de ser continuo como en el método de Montecarlo clásico, nuestra proporción μ queda sesgada por el espacio que hay entre dos posibles valores. Para contrarrestar esta situación, nosotros en vez de construir el intervalo de confianza como se comúnmente se calcula, a partir de la proporción que nos generó nuestra función Montecarlo, se evaluará varias veces la función hasta que la media $\bar{P}$ de los resultados de cada evaluación se establezca dado un error absoluto y relativo menor a e_R .

Dada una media $\bar{P}$ de simulaciones de Montecarlo y un error relativo máximo garantizado e_G , el intervalo de confianza utilizado en este proyecto se define como:

$$[\bar{P} - e_G \cdot \bar{P}, \bar{P} + e_G \cdot \bar{P}]$$

Este intervalo no se construye mediante el enfoque estadístico tradicional que utiliza el error estándar e_σ y un nivel de confianza explícito asociado al valor crítico Z , Sin embargo, se considera

un intervalo de confianza ya que cumple con la función principal de este tipo de intervalos: proporcionar un rango estimado dentro del cual se espera que se encuentre el valor verdadero del polinomio cromático, considerando la incertidumbre inherente del método.

El valor de e_G se calculará de manera empírica más adelante del desarrollo del proyecto, sin embargo, el uso de e_G como base del cálculo de los intervalos, permite que el intervalo se ajuste dinámicamente a la magnitud de las soluciones y al tamaño del problema, sin perder representatividad ni precisión.

El algoritmo para calcular los intervalos de confianza de la evaluación del polinomio cromático $P(G, k)$ de una gráfica $G = (V, E)$ con k colores, un error estándar un error relativo e_R y un nivel de confianza Z , utilizando el método de Montecarlo es el siguiente:

```
Intervalos(Grafica G, Colores k, Error rel.  $e_R$ , Nivel de confianza Z)
    vector<Flotante> v
    flotante media ← Montecarlo(G, k,  $e_R$ , Z), cabs, crel
    hacer
        flotante mant ← media(v)
        flotante r ← Montecarlo(G, k,  $e_R$ , Z)
        insertar r en v
        media ← media(v)
        cabs ← |media - mant|
        crel ← cabs / mant
    fin
    mientras(cabs >  $e_R$  & crel >  $e_R$ )
        regresa [media - media *  $e_G$ , media + media *  $e_G$ ]
fin
```

Para la implementación de los algoritmos se utilizó el lenguaje C++ en su versión del 2023 y el IDE Visual Studio Code. Estas implementaciones se describen en la siguiente subsección.

6.2. Módulo de coloraciones aleatorias

El módulo define una clase llamada `coloración`, diseñada para trabajar con gráficas y realizar operaciones relacionadas con la asignación de colores a los nodos. La clase incluye dos métodos principales: uno para generar una coloración aleatoria de una gráfica y otro para verificar la validez de esa coloración. Además, implementa un generador de números aleatorios basado en hardware para garantizar la calidad y aleatoriedad de las coloraciones generadas.

El método de generación de coloraciones utiliza la clase `std::uniform_int_distribution` de la biblioteca estándar `<random>`. Esta clase permite asignar colores a los nodos de forma uniforme, asegurando que cada color dentro del rango especificado tenga la misma probabilidad de ser elegido. Para la semilla del generador de distribución, se emplea la intrínseca `_rand64_step` de la biblioteca `<immintrin.h>`. Esta función genera números aleatorios de 64 bits directamente desde el hardware del procesador, aprovechando la instrucción `RDRAND` [13]. Este enfoque es particularmente importante porque la generación de coloraciones se realiza de manera concurrente, y el uso de un generador basado en hardware reduce significativamente el riesgo de conflictos o patrones repetitivos que podrían surgir con generadores pseudoaleatorios tradicionales como `srand` o `std::mt19937`. Al utilizar `_rand64_step`, cada hilo puede obtener números aleatorios seguros y de alta calidad sin depender de una semilla compartida o del reloj del sistema, como ocurre con `time(0)`, lo que hace que esta implementación sea adecuada para escenarios concurrentes. Además,

este enfoque garantiza que las coloraciones generadas sean verdaderamente aleatorias, incluso en ejecuciones paralelas intensivas.

El método de verificación de coloraciones evalúa si una asignación de colores es válida para una gráfica representada por una matriz de adyacencia. Para optimizar la verificación, el algoritmo recorre únicamente la parte triangular superior derecha de la matriz, bajo el supuesto de que la matriz de entrada está correctamente estructurada (simétrica y con valores booleanos).

6.3. Módulo de evaluación estadística

Este módulo nos sirve para tener las funciones estadísticas que se necesitan para el cálculo de los intervalos de confianza del método de Montecarlo. Este módulo está compuesto de una clase llamada `estadística` que tiene como atributos el número crítico Z , el error estándar deseado e_R y el número de vértices de la gráfica $|V|$, estas se inicializan mediante un constructor. Por otra parte, tiene como función privada el cálculo de la desviación estándar de una muestra y como función pública el cálculo del intervalo de confianza de un vector de datos y su media, cuyo retorno es un objeto `std::pair` de la biblioteca estándar `<utility>` que contiene los límites superior e inferior.

Todas las operaciones están diseñadas para soportar entradas y salidas de flotantes de hasta 128 bits y puede realizar los cálculos gracias al módulo que se describirá a continuación y la biblioteca `<quadmath.h>`.

6.4. Módulo de operaciones de números enteros y flotantes de 128 bits

Dado que estamos trabajando con un problema cuyos resultados crecen exponencialmente, el uso de tipos de datos enteros y flotantes comunes como `long long` o `long float` sea bastante limitado, puesto que para evaluaciones del polinomio cromático de gráficas de cierto número de vértices en adelante estos se desborden. Para solucionar esto y dar un espacio de soluciones precisas se utilizó el tipo de dato `__int128_t` para enteros de 128 bits [14], cuyo rango es de -2^{127} hasta $2^{127} - 1$ y el tipo de dato `std::float128_t` de la biblioteca `<stdfloat>`, que fue introducida en la versión C++23, para flotantes de 128 bits, cuyo rango de valores positivos es de aproximadamente un valor máximo de 1.18×10^{4932} hasta un mínimo de 1.18×10^{-4932} y puede guardar números enteros sin perder precisión de la mantisa de hasta 2^{113} [15].

Debido a que algunas funciones que se utilizan en este proyecto no están estandarizadas para este tipo de datos, este módulo tiene cuatro funciones, que son para calcular x^y para un x del tipo `__int128_t` o `std::float128_t` y una y del tipo `int` (Implementada como una alternativa sin pérdida de precisión del uso de `pow` de la biblioteca estándar de C++), el cálculo del valor absoluto de un `std::float128_t`, el error relativo (parte importante del módulo de evaluación estadística) de dos `std::float128_t` y, por último, dos funciones `to_string` que convierten un `__int128_t` o un `std::float128_t` a una cadena, puesto que estos tipos de datos no están estandarizados para la salida estándar.

6.5. Módulo adaptativo de simulaciones

Este módulo es el encargado de ejecutar el método de Montecarlo para una gráfica, consta de una clase llamada `monteCarlo` que contiene como atributos privados dos flotantes Z y e_{rel} que representan, respectivamente el nivel de confianza y el error relativo del método; además, una matriz de adyacencia G implementada con `std::vector<std::vector<bool>>` para que se pueda manipular fácilmente y un entero k que representa el número de colores con el que evaluaremos $P(G, k)$, estos atributos se inicializan mediante un constructor. Además de estos atributos, hay un objeto del tipo `coloración` del módulo de coloraciones aleatorias y un flotante *media* que se

utilizará posteriormente, estos dos atributos también son privados, aunque no se inicializan mediante el constructor.

La clase cuenta con tres funciones privadas. La primera es la función `montecarlo` que utiliza el algoritmo para calcular una evaluación del método de Montecarlo, para esto, utiliza del objeto `coloración` para calcular coloraciones aleatorias y para verificar si son propias o no. Puesto que el número de muestras puede llegar a ser muy grande, dependiendo del error relativo y del espacio de posibles coloraciones, se optó por utilizar hilos mediante la biblioteca `<thread>`, se implementó una forma de asignarle a todos los hilos que tiene el hardware (excepto uno), determinados mediante la función `std::thread::hardware_concurrency()`, la tarea de generar de manera concurrente un bloque de simulaciones a través la función privada `simulación`, que tiene como parámetros el tamaño del bloque de simulaciones que cada hilo va a realizar y un apuntador a una variable de tipo `std::atomic<int>` donde se irá sumando de manera concurrente si una coloración es válida o no. Ya acabado el algoritmo, la función devuelve un `std::float128_t` que contiene el resultado del método de Montecarlo. Por otro lado, cuenta con la función `mediamc` que retorna un `std::vector<std::float128_t>` donde se calcula la segunda parte del algoritmo, donde se manda a llamar a la función `montecarlo` hasta que la media de los resultados de las evaluaciones del método converja. cada evaluación del método que da la función `montecarlo` se guarda en un vector. Al final de la función, se le asigna a la variable `media` el resultado de la media de las evaluaciones y se retorna el vector.

Por último, se tiene como atributo público una estructura llamada `resultado` que tiene como atributos una tupla de tipo `std::pair<std::float128_t, std::float128_t>` donde se guardará el límite inferior y superior del intervalo y un `std::float128_t` que guardará la media de las evaluaciones. Además, se tiene la función pública `intervalo` que manda a llamar a la función `mediamc` para calcular el vector de resultados y la media, que posteriormente usa para crear el intervalo de confianza a partir de un objeto estadística del módulo de evaluación estadística, esta función regresa una estructura `resultado` con el intervalo y la media.

7. Resultados

Para poder probar la fiabilidad y escalabilidad del método, se generaron cuatro bloques de instancias para poner a prueba el proyecto, donde se tomará el tiempo del tiempo del algoritmo y se comparará si el intervalo de confianza tiene dentro el resultado real del polinomio. Habrá dos tipos de resultados reales, los calculados mediante el algoritmo de borrado-contracción implementado en el servicio social del alumno [8] y los que se calculan mediante fórmulas generales para cierto tipos de gráficas.

Para esta sección, el proyecto se ejecutó en una computadora con un procesador Intel® Core™ i5-8350U que tiene un procesador de cuatro núcleos con una frecuencia máxima de 3.6Ghz, ocho hilos lógicos y una memoria caché de 6MB [16]. Para compilar el proyecto, se utilizó GNU 14.2.0 [14] y se utilizaron las siguientes banderas: `-std=c++23`, `-march=native` y `-lquadmath`. Además, para calcular los tiempos de ejecución en segundos de los dos algoritmos se utilizó `std::chrono::high_resolution_clock` de la biblioteca `<chrono>`.

Los datos estadísticos que se utilizaron fue un error relativo $e_R = 0.01$, un intervalo de confianza del 99% con un $Z = 2.745$ y un error garantizado $e_G = 0.05$

A continuación, se describen las instancias y los resultados obtenidos con el proyecto.

7.1. Método de Montecarlo vs borrado-contracción.

Por el paquete de Nauty and Traces [17], se pudieron calcular todas las gráficas no isomorfas de hasta 8 vértices. Además, se calcularon de manera aleatoria, utilizando la misma semilla `_rdrand64_step` del módulo de coloraciones aleatorias, 100 gráficas de 9 y 10 vértices, 50 de 11 vértices y 10 de 12 vértices.

Para cada una de las gráficas, se generaron cinco instancias del polinomio cromático de estas, donde k varia desde el número de vértices de la gráfica (que nos asegura que el polinomio no sea cero) y se va aumentando en cinco. Estas instancias se pasarán como entrada de un programa que calculará el resultado utilizando los módulos del proyecto y un módulo que calcula el polinomio utilizando el algoritmo borrado-contracción [8].

Por la naturaleza exponencial del algoritmo de borrado-contracción, para $|V| > 12$ el tiempo para calcular el polinomio crece a desmedida, por lo tanto, para casos grades se utilizarán polinomio conocidos como se explica en la siguiente subsección.

En la tabla 1 se muestra la comparación de los resultados del algoritmo de borrado-contracción con los del método de Montecarlo.

Vértices	Rotal de gráficas	Total de evaluaciones	Montecarlos correctos
1	1	5	5
2	2	10	10
3	4	20	20
4	11	55	55
5	34	170	170
6	156	780	780
7	1044	5220	5219
8	12346	61730	61730
9	100	500	500
10	100	500	500
11	50	250	250
12	10	50	50
TOTAL:	13858	69290	99.99856%

Tabla 1

En la tabla 2 se muestra la comparación del tiempo medio que se tomó el algoritmo de borrado-contracción y el método de Montecarlo.

Vértices	Montecarlo (s)	Borrado-contracción (s)
1	0.046	0
2	0.1141	0
3	0.19165	5.00E-05
4	0.258564	5.45E-05
5	0.339565	0.000241176
6	0.389083	0.00127949
7	0.431614	0.00674138
8	0.474063	0.0350778

Vértices	Montecarlo (s)	Borrado-contracción (s)
9	0.546522	0.252268
10	0.596164	1.38836
11	0.651544	10.5609
12	0.6879	93.7336

Tabla 2

7.2. Método de Montecarlo vs evaluaciones de polinomios conocidos

7.2.1. Método de Montecarlo para gráficas completas

Una gráfica completa es una gráfica $G = (V, E)$ en la cual existe una arista $e \in E$ para cada par ordenado de vértices $\{u, v\} \in V$. El polinomio cromático $P(G, k)$ de una gráfica completa es:

$$P(G, \lambda) = \prod_{i=0}^{|V|-1} (\lambda - i)$$

Para nuestro siguiente bloque de instancias, se calcularán cinco evaluaciones del polinomio cromático de una gráfica de n vértices, con $1 \leq n \leq 20$ y $2n \leq k \leq 2n + 25$. Esto para determinar la fiabilidad del método, puesto que este tipo de gráficas son las que tienen la proporción μ_γ más pequeña con respecto a las demás gráficas, esto debido a que una gráfica completa de $|V|$ vértices tiene la cantidad de aristas $|E|$ más grande, por lo que hay más restricciones para hacer una coloración válida.

En la tabla 2 se muestra la tabla del número de vértices de la gráfica completa y el tiempo medio que tomó el algoritmo del Montecarlo por tamaño de la gráfica:

Vértices	Tiempo (s)	Vértices	Tiempo (s)
1	0.0447492	12	0.578634
2	0.0891044	13	0.829246
3	0.134677	14	0.923199
4	0.178522	15	1.12615
5	0.22989	16	0.994436
6	0.276829	17	1.39664
7	0.317714	18	1.43728
8	0.438226	19	1.61296
9	0.436527	20	2.27559
10	0.48288		
11	0.675002		

Tabla 3

La salida del programa también nos muestra el número de evaluaciones correctas que resolvió el método de Montecarlo:

Evaluaciones: 100 Montecarlos correctos: 99
--

7.2.2. Método de Montecarlo para gráficas sin aristas

Una gráfica sin aristas es una gráfica $G = (V, E)$ donde E no contiene elementos. El polinomio cromático $P(G, \lambda)$ de una gráfica sin aristas es:

$$P(G, \lambda) = \lambda^{|V|}$$

La particularidad de este tipo de gráficas es que cualquier coloración γ es una coloración propia de G , por lo que, para observar mejor la complejidad del algoritmo (puesto que el generar la coloración aleatoria es de tiempo $O(|V|)$ y toma el tiempo suficiente para sentir que en las primeras evaluaciones la complejidad del algoritmo, cuando no es así), se utilizó una coloración predeterminada. Se calcularon 200 polinomios cromáticos de gráficas de 1 hasta 200 vértices, con un número de colores $k = 1$ para evitar que el resultado se desborde.

A continuación, en la tabla 4 se resume el tiempo que tardó el método de Montecarlo para las gráficas sin aristas, con una coloración predeterminada:

Vértices	Tiempo (s)
1	0.0035395
10	0.0141965
20	0.0566053
30	0.110834
40	0.191658
50	0.30731
60	0.446971
70	0.72875
80	0.950191
90	1.18204
100	1.47208
200	5.81146

Tabla 4

En la sección de análisis de resultados se generará un gráfico de dispersión donde está contenida la información de las 200 coloraciones, esta tabla se resumió para no ocupar demasiado espacio. La salida del programa también nos muestra el número de evaluaciones correctas que resolvió el método de Montecarlo:

Evaluaciones: 200 Montecarlos correctos: 199

7.2.3. Método de Montecarlo para caminos

Un camino es una gráfica $G = (V, E)$ donde, para cada vértice $v_i \in V$ etiquetado de 0 a $|V| - 1$, se tiene una arista $e = \{v_i, v_{i+1}\} \in E$. El polinomio cromático $P(G, \lambda)$ de un camino expresado como una gráfica G es:

$$P(G, k) = \lambda(\lambda - 1)^{|V|-1}$$

Este tipo de gráficas, además de tener un polinomio conocido y una matriz de adyacencia fácil de calcular, nos da una buena base para poder expresar la evaluación del polinomio cromático más grande que puede almacenar el hardware sin desbordarse. Para este bloque de instancias, para caminos de $1 \leq |V| \leq 30$ vértices, se calcularon cinco evaluaciones del polinomio cromático de por cada camino con $5 \leq k \leq 13$ colores.

En tabla se muestra el número de vértices del camino y el tiempo medio del método de Montecarlo por tamaño de la gráfica:

Vértices	Tiempo (s)	Vértices	Tiempo (s)
1	0.044787	17	1.14237
2	0.0902731	18	1.38257
3	0.133712	19	1.62379
4	0.181303	20	1.15114
5	0.25428	21	1.549
6	0.282938	22	1.68706
7	0.351319	23	2.1722
8	0.395523	24	1.56711
9	0.39447	25	2.721
10	0.436692	26	1.82194
11	0.799	27	1.95506
12	0.703344	28	2.94523
13	0.793626	29	3.61092
14	1.08502	30	2.60713
15	0.946405		
16	1.10909		

Tabla 5

La salida del programa también nos muestra el número de evaluaciones correctas que resolvió el método de Montecarlo:

Evaluaciones: 150 Montecarlos correctos: 144

La evaluación más grande que pudo hacer el proyecto en todos las instancias de prueba de este documento, sin desbordarse, fue la del polinomio cromático de un camino de 30 vértices y 13 colores, a continuación, se muestra la salida de esta evaluación:

P(G, 13) = 257157673283083956856362815717376 Usando el método de Montecarlo: Intervalo de confianza: [242352758229274891168832907691354.312500, 267863574884988039098325636299058.437500] Media: 255108166557131465133579271995206.375000 Tiempo: 1.42259 Error relativo: 0.007970
--

7.2.4. Peor de los casos del método de Montecarlo

Debido a que nuestro espacio de la muestra aumenta dinámicamente para tener un mejor espacio de soluciones, el peor de los casos es bastante intuitivo: Cuando tenemos una evaluación del polinomio de una gráfica G con k colores donde $P(G, k) = 0$, el algoritmo va a aumentar el espacio de la muestra hasta que $n \leq k^{|V|}$, esto para asegurarse que no está dejando una coloración propia de lado. Solo en este caso específico, se obtiene el peor de los casos del algoritmo y tendrá una complejidad temporal exponencial $O(k^{|V|})$.

Para demostrar esto, el polinomio cromático de una gráfica de $|V|$ vértices con evaluación $P(G, k) = 0$ con el mayor k posible, sin tener en cuenta gráficas con lazos, es la evaluación del polinomio cromático de una gráfica completa de $|V|$ vértices con $k = |V| - 1$ colores. Se evaluaron las gráficas completas de hasta 10 colores con esta k para ver cómo cambiaba el tiempo con respecto al número de vértices del peor de los casos.

En la tabla 5 se muestra el tiempo medio del método de Montecarlo en el peor de los casos con respecto al número de vértices de la gráfica, en los resultados, de los 10 casos de prueba, los 10 casos se resolvieron correctamente:

Vértices	Tiempo (s)
1	0.0039302
2	0.0051489
3	0.0032547
4	0.0032439
5	0.0040642
6	0.002902
7	0.117013
8	1.99695
9	37.0411
10	1107.33

Tabla 6

8. Análisis de los resultados

8.1. Cálculo del error garantizado e_G para el intervalo de confianza

Con base en los resultados de la sección anterior, se tiene que, de un total de 69750 instancias en total, se resolvieron 69741 intervalos de confianza de manera correcta. El algoritmo, como se mencionó al inicio de la sección 7, se implementó con un error garantizado de $e_G = 0.05$ el cual, a la hora de aplicarlo a las instancias, resulta en más del 99% de intervalos correctos, que es lo que se espera con el nivel de confianza dado. Este e_G puede ser considerado y calculado de diferentes formas, pero en este proyecto se calculó a partir la media del error relativo de la media del método de Montecarlo y el resultado real de las instancias de la sección 7.1 de este documento.

8.2. Análisis del método de Montecarlo vs borrado-contracción.

La comparación entre el método de Montecarlo y el algoritmo de borrado-contracción revela que, aunque ambos son precisos para evaluar el polinomio cromático, sus características de rendimiento divergen significativamente al aumentar la complejidad de las gráficas. El método de

Montecarlo destaca como una alternativa eficiente y escalable, especialmente para gráficas grandes, mientras que el borrado-contracción presenta limitaciones debido a su complejidad exponencial inherente.

En la Tabla 1, se observa que el método de Montecarlo alcanzó una precisión de 99.99856%, lo que evidencia su fiabilidad para estimar los valores exactos calculados mediante borrado-contracción. Este porcentaje alto se justifica por la correcta configuración del intervalo de confianza y el uso de técnicas adaptativas que ajustan la cantidad de simulaciones en función de la gráfica analizada. Además, como se muestra en la Ilustración 1, aumenta de manera exponencial el algoritmo de borrado-contracción el tiempo de ejecución cuando aumenta el número de vértices de las gráficas, mientras el método de Montecarlo se mantiene de manera casi lineal.

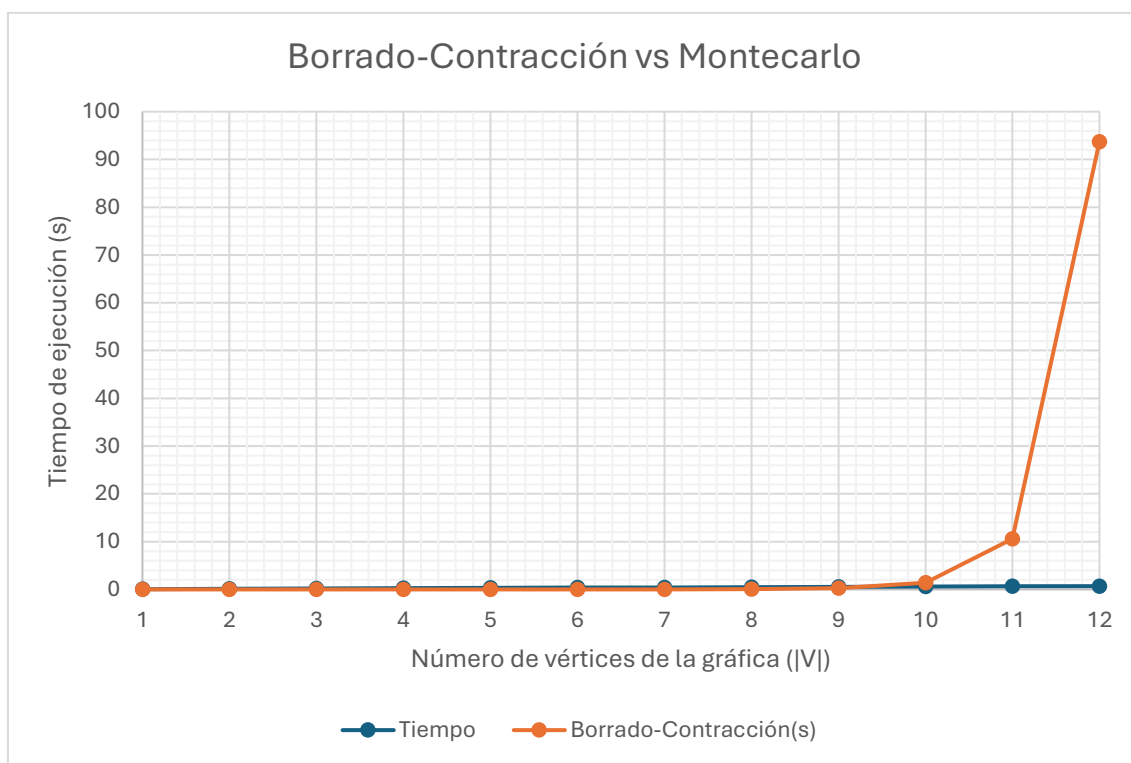


Ilustración 1

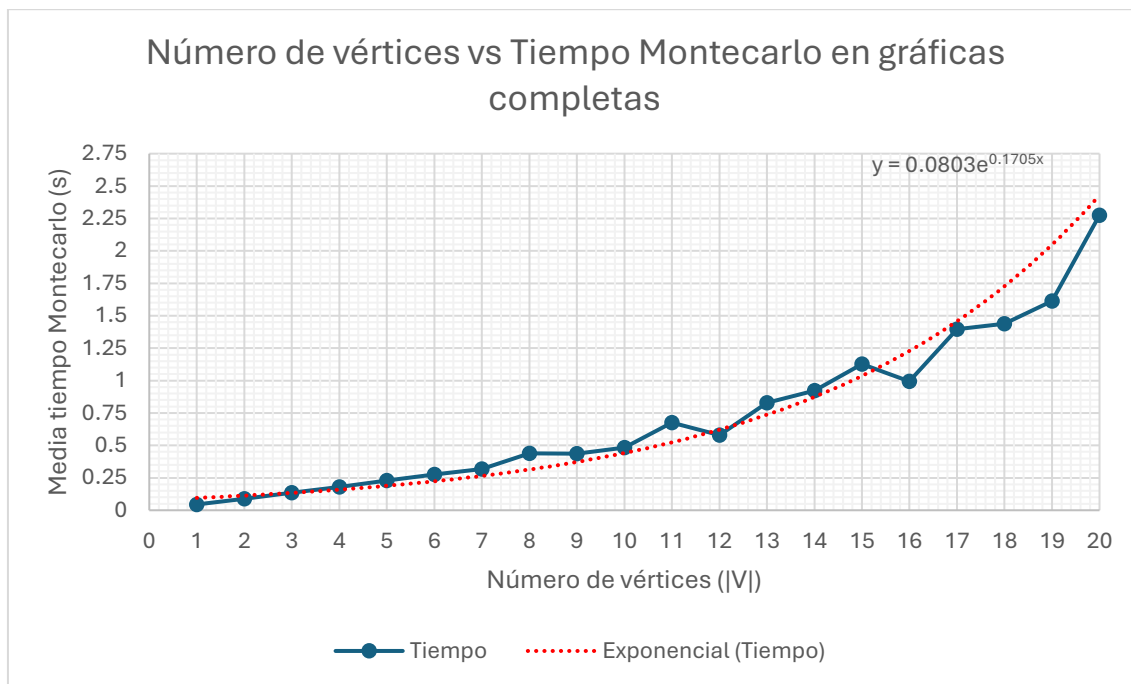
8.2. Análisis del método de Montecarlo vs evaluaciones de polinomios conocidos

8.2.2. Análisis del método de Montecarlo para gráficas completas

Para gráficas completas, donde cada nodo está conectado con todos los demás, el método de Montecarlo destacó por su capacidad para evaluar polinomios cromáticos en escenarios con restricciones máximas. Estas gráficas presentan un desafío considerable debido al bajo número relativo de coloraciones válidas en comparación con el total posible. A pesar de ello, el método mantuvo una precisión del **99%**, como se muestra en la Tabla 3, lo que confirma su adaptabilidad a estas condiciones.

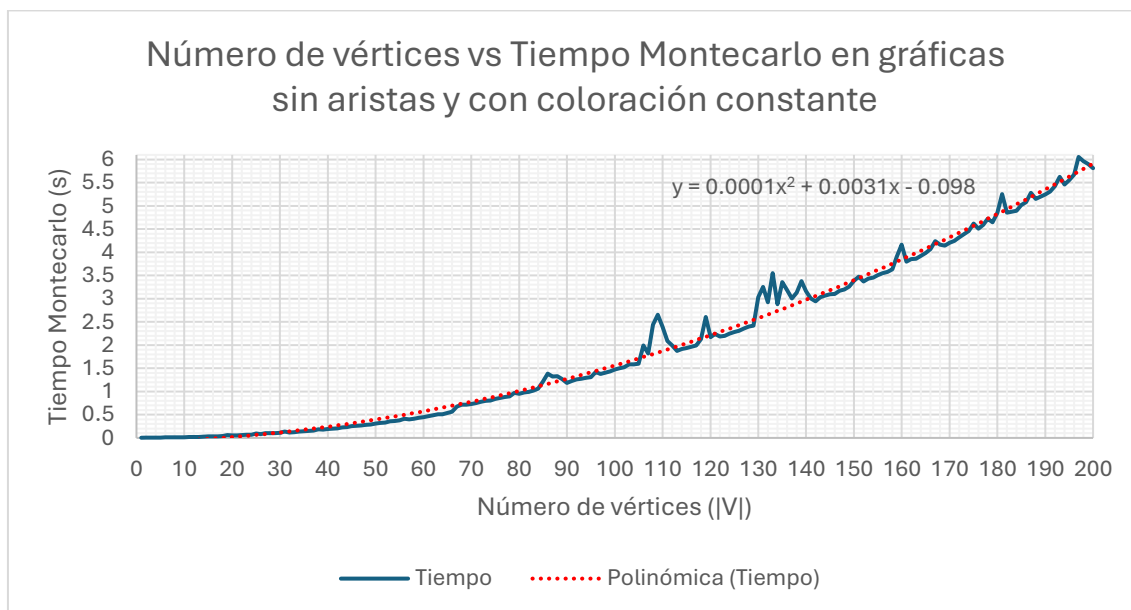
Los tiempos de ejecución crecieron moderadamente con el número de vértices, reflejando la complejidad intrínseca del problema, pero sin alcanzar niveles inaceptables gracias al enfoque probabilístico.

En la Ilustración 2 se muestra un gráfico de la media del tiempo que tomó el método de Montecarlo con respecto al número de vértices y una función exponencial a partir de interpolación numérica.



8.2.3. Análisis del método de Montecarlo para gráficas sin aristas

El método de Montecarlo demostró ser eficiente para gráficas sin aristas, ya que cualquier coloración es válida. En la Ilustración 3 se evidencia un crecimiento cuadrático respecto al número de vértices, que es la complejidad general del algoritmo.



8.2.4. Análisis del método de Montecarlo para caminos

Para gráficas de tipo camino, Montecarlo mantuvo una alta precisión con 144 evaluaciones correctas de un total de 150. Estas gráficas presentan una complejidad intermedia, ya que las restricciones dependen de las conexiones lineales entre los nodos. Los tiempos de ejecución, como se muestra en la Ilustración 4, tienden a un comportamiento cuadrático, atribuible a la carga adicional de generación y validación de coloraciones propias en caminos más largos.

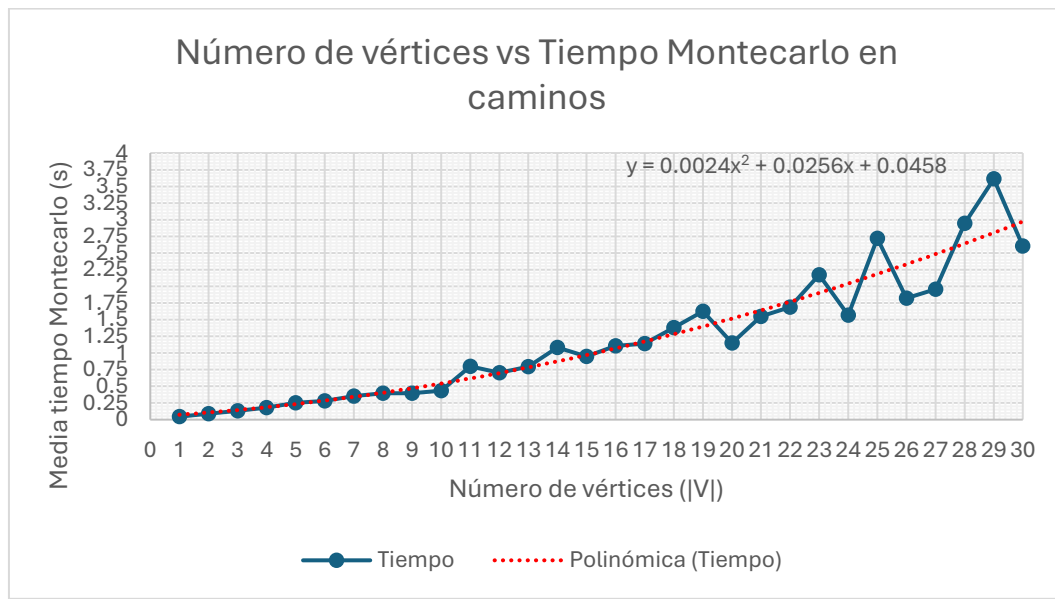


Ilustración 4

8.2.5. Análisis del peor de los casos del método de Montecarlo

La Tabla 6 reportó un tiempo superior a 1107 segundos para una gráfica completa de 10 vértices, evidenciando los límites prácticos del método bajo condiciones extremas. Esto es coherente con la necesidad de incrementar significativamente el tamaño de la muestra para garantizar resultados confiables. La Ilustración 5 muestra más a detalle esta complejidad máxima del algoritmo.

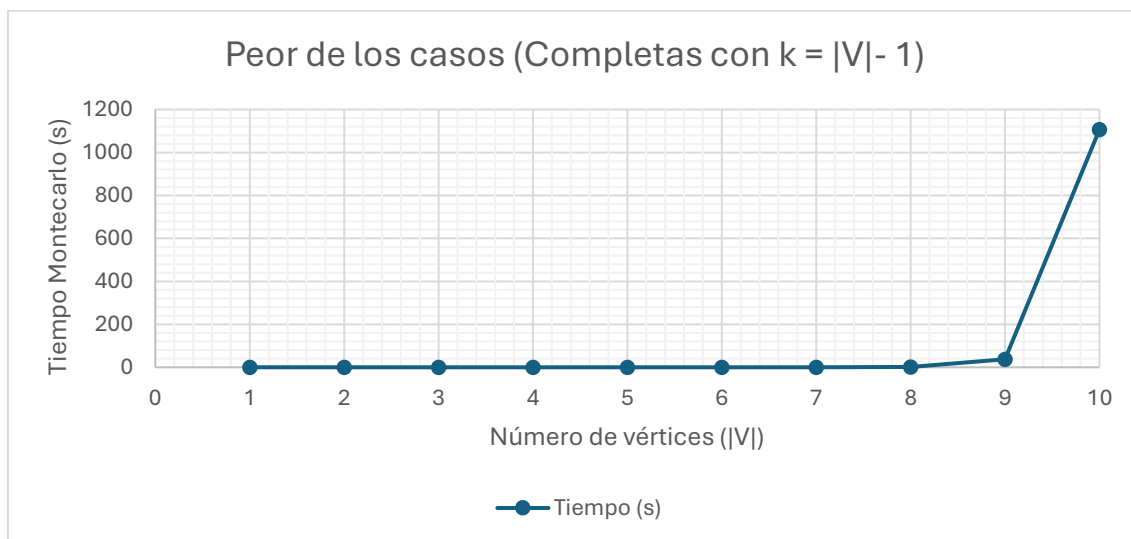


Ilustración 5

8.3. Conclusión de análisis de resultados

- El método de Montecarlo es una alternativa escalable y eficiente frente al algoritmo de borrado-contracción, especialmente para gráficas grandes.
- Su precisión es alta en todos los casos analizados, y los intervalos de confianza generados contienen los valores exactos en más del 99% de las evaluaciones.
- Una de sus principales limitantes es el cálculo estadístico, que es algo intrínseco del método, ya que, cuando hay un espacio de posibles coloraciones tan grande con una muy pequeña proporción de coloraciones propias, el algoritmo tenderá a fallar.
- El tiempo de ejecución tiende a una complejidad cuadrática para todas las gráficas y colores en general y tiende a ser exponencial, que es el mismo tipo de complejidad que el mejor algoritmo exacto existente, en el peor de sus casos, que es cuando $P(G, k) = 0$ o está muy cerca de este.

9. Conclusiones

Este proyecto demuestra la viabilidad y efectividad del método de Montecarlo para la estimación aproximada de la evaluación polinomio cromático $P(G, k)$ de una gráfica G con k colores, proporcionando una solución práctica a un problema #P-completo que resulta intratable mediante métodos exactos para gráficas de gran tamaño. A través de la implementación de los módulos principales del proyecto se logró desarrollar un sistema robusto capaz de generar estimaciones confiables con un control riguroso del error y la incertidumbre mediante intervalos de confianza.

Gracias a la necesidad de gestionar un espacio aleatorio de muestras de gran tamaño, inherente al método de Montecarlo, y de lidiar con resultados de magnitudes extremadamente grandes, este proyecto exigió una exploración profunda de alternativas avanzadas en programación y optimización de los recursos. La implementación no solo requirió una comprensión detallada de los principios teóricos detrás de Montecarlo, sino también un enfoque práctico orientado a maximizar el rendimiento del hardware disponible y a aprovechar al máximo las capacidades de la biblioteca estándar de C++. El proyecto requirió la integración de técnicas como gestión eficiente de memoria, el uso de bibliotecas intrínsecas del hardware para generar números aleatorios y el uso de hilos, mostrando que abordar problemas combinatorios de esta magnitud implica una combinación de conocimiento teórico, habilidades prácticas y adaptabilidad tecnológica. En este sentido, la implementación no solo fue un componente técnico, sino una parte esencial que permitió transformar un problema abstracto en un sistema funcional y escalable, capaz de generar resultados precisos en escenarios que anteriormente parecían inalcanzables.

Por último, este proyecto valida la utilidad del método de Montecarlo más allá del cálculo del polinomio cromático, destacando su potencial para abordar otros problemas #P-completos, los cuales comparten características similares, como el crecimiento exponencial del espacio de soluciones y la dificultad de aplicar métodos deterministas. Este trabajo no solo representa un avance en la comprensión y solución de problemas combinatorios, sino que también establece un marco metodológico sólido para futuros estudios y aplicaciones en el campo de los métodos heurísticos y el cómputo científico.

10. Referencias Bibliográficas

- [1] N. Polson y V. Sokolov, «Counting N Queens», 11 Julio 2024. [En línea]. Available: <https://arxiv.org/pdf/2407.08830>. [Último acceso: 2025 Enero 2025].
- [2] C. Zhang y M. Jianpeng, «Counting solutions for the N-queens and Latin-square problems by Monte Carlo simulations», *Physical Review E*, vol. 79, n° 1, p. 016703, 2009.
- [3] A. Jensen y I. Beichl, «A Sequential Importance Sampling Algorithm for Counting Linear Extensions», *ACM J. Exp. Algorithmics*, vol. 25, n° 1.7, pp. 1-14, 2020.
- [4] R. M. Karp, M. Luby y N. Madras, «Monte-Carlo approximation algorithms for enumeration problems», *Journal of Algorithms*, vol. 89, n° 10, pp. 429-448, 1989.
- [5] J. L. Chacón, Introducción a la Teoría de Grafos, Granada: Universidad de Granada, 2005.
- [6] R. P. Grimaldi, Matemáticas discretas y combinatoria. Una introducción con aplicaciones, Delaware: Addison-Wesley Iberoamericana, 1997.
- [7] J. A. Bondy y U. S. R. Murty, Graph Theory, New York: Springer, 2008.
- [8] L. D. Ramos y M. G. Rodríguez, «Calculadora Cromática», Universidad Autónoma Metropolitana Unidad Azcapotzalco, 2 Junio 2023. [En línea]. Available: <https://github.com/danielramosl/CalculadoraCromatica>. [Último acceso: 2025 01 09].
- [9] H. S. Wilf, Algorithms and Complexity, Pennsylvania: University of Pennsylvania, 1994.
- [10] R. E. Walpole, S. L. Myers y Y. Keying, Probabilidad y estadística para ingeniería y ciencias, México: Pearson Education, 2012.
- [11] O. J. Suárez, Métodos Monte Carlo y Productos Estructurados, Guanajuato: Centro de Investigación en Matemáticas, A.C., 2012.
- [12] P. Saavedra y V. H. Ibarra, El método Monte-Carlo y su aplicación a las finanzas, México: UAM Iztapalapa, 2008.
- [13] Intel Corporation, Intel® 64 and IA-32 Architectures Software Developer's Manual, California: Intel Corporation, 2023.
- [14] Richard M. Stallman & GCC Developer Community, Using the GNU Compiler Collection (For GCC version 14.2.0), Massachusetts: GNU Press, 2024.
- [15] C++Reference, «Tipos de punto flotante de anchura fija (desde C++23)», cppreference, 5 Julio 2024. [En línea]. Available: <https://es.cppreference.com/w/cpp/types/floating-point>. [Último acceso: 2025 Enero 07].
- [16] Intel Corporation, «Procesador Intel® Core™ i5-8350U», [En línea]. Available: <https://www.intel.la/content/www/xl/es/products/sku/124969/intel-core-i58350u-processor-6m-cache-up-to-3-60-ghz/specifications.html>. [Último acceso: 2025 Enero 12].
- [17] B. McKay y A. Piperno, «Practical Graph Isomorphism», *Journal of Symbolic Computation*, vol. II, n° 60, pp. 94-112, 2014.

12. Entregables

12.1. Pseudocódigo de los algoritmos

```
g(Gráfica G, Coloración  $\gamma$ )
  para  $i \leftarrow 0$  mientras  $i < |V|$  hacer
    para  $j \leftarrow i + 1$  mientras  $j < |V|$  hacer
      si  $G[i][j] = 0 \ \& \ (\gamma(i) = \gamma(j))$ 
        regresa falso
      fin
       $j \leftarrow j + 1$ 
    fin
     $i \leftarrow i + 1$ 
  fin
  regresa verdadero
fin
```

Función para determinar si una coloración γ es una coloración propia para una gráfica G .

```
Montecarlo(Grafica  $G$ , Colores  $k$ , Error rel.  $e_R$ , Nivel de confianza  $Z$ )

  flotante  $f \leftarrow 1$ ,  $e_\mu \leftarrow 1$ ,  $\mu_\gamma \leftarrow 0.5$ 
  hacer
    entero  $n \leftarrow (f \cdot Z^2 \cdot \mu_\gamma \cdot (1 - \mu_\gamma)) / e_R^2$ 
    entero val  $\leftarrow 0$ 
    desde ( $i \leftarrow 0$ ) hasta  $n$ 
      hacer
        Coloración  $\gamma \leftarrow \text{coloraciónAleatoria}(G, k)$ 
        val  $\leftarrow \text{val} + g(G, \gamma)$ 
         $i \leftarrow i + 1$ 
      fin
    flotante  $\mu_{act} \leftarrow \text{val} / n$ 
    si ( $\mu_{act} = 0$ )
      hacer
        sí ( $n > k^{|V|}$ ) entonces
          regresa 0
        fin
         $f \leftarrow 2 \cdot f$ 
        continuar
    fin
     $e_\mu \leftarrow \sqrt{\mu_\gamma \cdot (1 - \mu_\gamma) / n}$ 
    si ( $e_\mu > e_R$ )
      hacer
         $f \leftarrow f \cdot ((1 + (e_\mu - e_R)) / e_R)$ 
      fin
     $\mu_\gamma \leftarrow \mu_{act}$ 
  fin
  mientras ( $e_\mu > e_R$ )
    regresa  $\mu_\gamma \cdot k^{|V|}$ 
fin
```

Función para una simulación de Montecarlo

```

Intervalos(Grafica  $G$ , Colores  $k$ , Error rel.  $e_R$ , Nivel de confianza  $Z$ )
    vector<Flotante> v
    flotante media  $\leftarrow$  Montecarlo( $G$ ,  $k$ ,  $e_R$ ,  $Z$ ), cabs, crel
    hacer
        flotante mant  $\leftarrow$  media( $v$ )
        flotante r  $\leftarrow$  Montecarlo( $G$ ,  $k$ ,  $e_R$ ,  $Z$ )
        insertar r en v
        media  $\leftarrow$  media( $v$ )
        cabs  $\leftarrow$  |media - mant|
        crel  $\leftarrow$  cabs / mant
    fin
    mientras(cabs >  $e_R$  & crel >  $e_R$ )
        regresa [media - media *  $e_G$ , media + media *  $e_G$ ]
fin

```

Función para la creación del intervalo de confianza.

12.2. Código fuente del módulo de generación de coloraciones en C++

```

1. #pragma once
2. #include <immintrin.h>
3. #include <limits>
4. #include <random>
5. #include <vector>
6.
7. class coloración {
8.     private:
9.         struct random_rand {
10.             using result_type = uint64_t;
11.             uint64_t operator()() {
12.                 uint64_t res;
13.                 while (_rdrand64_step(reinterpret_cast<unsigned long long*>(&res)) == 0) {
14.                     continue;
15.                 }
16.                 return res;
17.             }
18.             static constexpr result_type min() {
19.                 return std::numeric_limits<result_type>::min( );
20.             }
21.             static constexpr result_type max() {
22.                 return std::numeric_limits<result_type>::max( );
23.             }
24.         };
25.
26.         random_rand gen;
27.
28.     public:
29.         std::vector<int> coloraciónAleatoria(int n, int c) {
30.             std::vector<int> arr(n);
31.             std::uniform_int_distribution<int> distribución(0, c - 1);
32.             for(int i = 0; i < n; ++i) {
33.                 arr[i] = distribución(gen);
34.             }
35.             return arr;
36.         }
37.
38.         bool verificarColoración(const std::vector<std::vector<bool>>& g, const std::vector<int>&
colores) {
39.             for(int i = 0; i < g.size(); ++i) {
40.                 for(int j = i + 1; j < g.size(); ++j) {
41.                     if(g[i][j] && colores[i] == colores[j]) {
42.                         return false;

```

```

43.         }
44.     }
45. }
46.     return true;
47. }
48. };
49.

```

Módulo “coloraciones.h”.

12.3. Código fuente del módulo de evaluación estadística en C++

```

1. #pragma once
2. #include "ops128.h"
3. #include <quadmath.h>
4. #include <stdfloat>
5. #include <utility>
6. #include <vector>
7.
8. class estadística {
9.     private:
10.         std::float128_t z;
11.         std::float128_t e;
12.         std::float128_t n;
13.
14.         std::float128_t desv(const std::vector<std::float128_t>& d, std::float128_t med) {
15.             std::float128_t res = 0;
16.             for(int i = 0; i < d.size(); ++i) {
17.                 res += potencia(d[i] - med, 2);
18.             }
19.             return sqrtq(res / std::float128_t(d.size() - 1));
20.         }
21.
22.     public:
23.         estadística(std::float128_t z, std::float128_t e, std::float128_t n) {
24.             this->z = z;
25.             this->e = e;
26.             this->n = n;
27.         }
28.
29.         std::pair<std::float128_t, std::float128_t> intervalo(const std::vector<std::float128_t>&
datos, std::float128_t media) {
30.             std::float128_t de = desv(datos, media);
31.             std::float128_t em = 0.05;
32.             return std::pair<std::float128_t, std::float128_t> (media - media * em, media + media
* em);
33.         }
34.     };
35.

```

Módulo “estadística.h”.

12.4. Código fuente del módulo de operaciones de 128 bits en C++

```

#pragma once
1. #include <algorithm>
2. #include <stdfloat>
3. #include <string>
4.
5. std::float128_t potencia(std::float128_t x, int n) {
6.     std::float128_t res = 1;
7.     for(int i = 0; i < n; ++i) {
8.         res *= x;
9.     }
10.     return res;

```

```

11. }
12.
13. __int128_t potencia(int x, int n) {
14.     __int128_t res = 1;
15.     for(int i = 0; i < n; ++i) {
16.         res *= x;
17.     }
18.     return res;
19. }
20.
21. std::float128_t absoluto(std::float128_t n) {
22.     return (n > 0) ? n : -n;
23. }
24.
25. std::float128_t erel(std::float128_t exc, std::float128_t sim) {
26.     return absoluto(exc - sim) / exc;
27. }
28.
29. std::string i128to_string(__uint128_t n) {
30.     if (n == 0) return "0";
31.     std::string res;
32.     while (n > 0) {
33.         res.push_back('0' + (n % 10));
34.         n /= 10;
35.     }
36.     std::reverse(res.begin(), res.end());
37.     return res;
38. }
39.
40. std::string f128to_string(std::float128_t n) {
41.     char res[200];
42.     quadmath_snprintf(res, sizeof res, "%Qf", n);
43.     return res;
44. }
45.

```

Módulo “ops128.h”.

12.5. Código fuente del módulo adaptativo de simulaciones en C++

```

1. #pragma once
2. #include <atomic>
3. #include <quadmath.h>
4. #include <thread>
5. #include <stdfloat>
6. #include <utility>
7. #include <vector>
8. #include "coloración.h"
9. #include "estadística.h"
10. #include "ops128.h"
11.
12. class monteCarlo {
13.     private:
14.         std::float128_t z;
15.         std::float128_t e;
16.         std::vector<std::vector<bool>> g;
17.         int c;
18.         coloración col;
19.         std::float128_t media;
20.
21.     public:
22.         monteCarlo(std::float128_t z, std::float128_t e, const std::vector<std::vector<bool>>&
g, int c) {
23.             this->z = z;
24.             this->e = e;

```

```

25.         this->g = g;
26.         this->c = c;
27.     }
28.
29. private:
30. void simulación(int bloqueTam, std::atomic<int>& validas) {
31.     for (int i = 0; i < bloqueTam; ++i) {
32.         std::vector<int> coloración = col.coloraciónAleatoria(g.size(), c);
33.         if (col.verificarColoración(g, coloración)) {
34.             ++validas;
35.         }
36.     }
37. }
38.
39. std::float128_t montecarlo() {
40.     std::float128_t p = 0.5;
41.     std::float128_t ep = 1;
42.     std::float128_t factor = 1;
43.     do {
44.         __int128_t n = factor * (potencia(z, 2) * p * (1 - p)) / (potencia(e, 2));
45.         std::atomic<int> validas(0);
46.         int nhilos = std::thread::hardware_concurrency() - 1;
47.         int tam = n / nhilos;
48.         int resto = n % nhilos;
49.         std::vector<std::thread> hilos;
50.         for (int i = 0; i < nhilos; ++i) {
51.             int bloqueTam = tam + (i == nhilos - 1 ? resto : 0);
52.             hilos.emplace_back(&monteCarlo::simulación, this, bloqueTam,
std::ref(validas));
53.         }
54.         for (auto& hilo : hilos) {
55.             hilo.join();
56.         }
57.         std::float128_t pact = std::float128_t(validas) / std::float128_t(n);
58.         if (pact == 0) {
59.             if (potencia(c, g.size()) < n) {
60.                 return 0;
61.             }
62.             factor *= 2;
63.             continue;
64.         }
65.         p = pact;
66.         ep = sqrtq(p * (1 - p) / n);
67.         if (ep > e) {
68.             factor *= (1 + (ep - e) / e);
69.         }
70.     } while (ep > e);
71.     return p * potencia(c, g.size());
72. }
73.
74. std::vector<std::float128_t> mediamc() {
75.     media = montecarlo();
76.     std::float128_t cabs, crel;
77.     int i = 1;
78.     std::vector<std::float128_t> datos = {media};
79.     do {
80.         std::float128_t actual = montecarlo();
81.         std::float128_t manterior = media / std::float128_t(i++);
82.         media += actual;
83.         datos.push_back(actual);
84.         cabs = absoluto(media / std::float128_t(i) - manterior);
85.         crel = cabs / manterior;
86.     } while (cabs > e && crel > e);
87.     media /= std::float128_t(i);
88.     return datos;

```



```

89.     }
90.
91.     public:
92.     struct resultado {
93.         std::pair<std::float128_t, std::float128_t> intervalo;
94.         std::float128_t media;
95.     };
96.
97.     resultado intervalo() {
98.         estadística est(z, e, std::float128_t(g.size()));
99.         std::vector<std::float128_t> d = mediamc();
100.         return {est.intervalo(d, media), media};
101.     }
102. };
103.

```

Módulo “montecarlo.h”.

12.6. Código fuente del módulo del algoritmo de Borrado-Contracción en C++

```

1. #include "ops128.h"
2. #include <string>
3. #include <vector>
4.
5. class polinomio {
6.     private:
7.     struct arista {
8.         int u;
9.         int v;
10.    };
11.    std::string sPolinomio;
12.    std::vector<std::vector<bool>> g;
13.    std::vector<int> fun;
14.
15.    bool no_arista(const std::vector<std::vector<bool>>& mat) {
16.        for(int i = 0; i < mat.size(); ++i) {
17.            for(int j = i + 1; j < mat.size(); ++j) {
18.                if(mat[i][j]) {
19.                    return false;
20.                }
21.            }
22.        }
23.        return true;
24.    }
25.
26.    arista primer_arista(const std::vector<std::vector<bool>>& mat) {
27.        for(int i = 0; i < mat.size(); ++i) {
28.            for(int j = i + 1; j < mat.size(); ++j) {
29.                if(mat[i][j] == 1) {
30.                    return arista {i, j};
31.                }
32.            }
33.        }
34.        return arista {0,0};
35.    }
36.
37.    std::vector<std::vector<bool>> borrado(std::vector<std::vector<bool>> mat, arista
arista) {
38.        mat[arista.u][arista.v] = 0; mat[arista.v][arista.u] = 0;
39.        return mat;
40.    }
41.
42.    std::vector<std::vector<bool>> contracción(const std::vector<std::vector<bool>>& mat,
arista arista) {
43.        std::vector<std::vector<bool>> res;

```

```

44.     for(int i = 0; i < mat.size(); ++i) {
45.         if(i == arista.u || i == arista.v) {
46.             continue;
47.         }
48.         std::vector<bool> arr;
49.         for(int j = 0; j < mat.size(); ++j) {
50.             if(j != arista.u && j != arista.v) {
51.                 arr.push_back(mat[i][j]);
52.             }
53.         }
54.         res.push_back(arr);
55.     }
56.
57.     int ini = 0;
58.     for(int i = 0; i < mat.size(); ++i) {
59.         if(i != arista.u && i != arista.v) {
60.             res[ini++].push_back(mat[i][arista.u] | mat[i][arista.v]);
61.         }
62.     }
63.
64.     res.push_back(std::vector<bool>());
65.     int size = res.size() - 1;
66.     for(int j = 0; j < mat.size(); ++j) {
67.         if(j != arista.u && j != arista.v) {
68.             res[size].push_back(mat[arista.u][j] | mat[arista.v][j]);
69.         }
70.     }
71.     res[size].push_back(0);
72.     return res;
73. }
74.
75. std::string cromático(const std::vector<std::vector<bool>>& mat) {
76.     if(no_arista(mat)) {
77.         return "x" + std::to_string(mat.size());
78.     } else {
79.         arista arista = primer_arista(mat);
80.         return cromático(borrado(mat, arista)) + "-" + cromático(contracción(mat,
arista)) + ")";
81.     }
82. }
83.
84. std::vector<int> traductor(int n, std::string fun) {
85.     std::vector<int> res(n + 1, 0);
86.     int signo = 1;
87.     std::vector<int> ant;
88.     for(int i = 0; i < fun.size(); ++i) {
89.         if(fun[i] == 'x') {
90.             std::string num = "";
91.             while(std::isdigit(fun[i + 1])) {
92.                 num += fun[i + 1];
93.                 ++i;
94.             }
95.             res[std::stoi(num)] += signo;
96.         } else if(fun[i] == '-') {
97.             ant.push_back(-1);
98.             signo *= -1;
99.             ++i;
100.        } else if(fun[i] == ')') {
101.            signo *= ant.back(); ant.pop_back();
102.        }
103.    }
104.    return res;
105. }
106.
107. __int128_t evaluación(const std::vector<int>& f, int k) {

```

```

108.     __int128_t res = 0;
109.     for(int i = 0; i < f.size(); ++i) {
110.         res += f[i] * potencia(k, i);
111.     }
112.     return res;
113. }
114.
115. public:
116.     polinomio(const std::vector<std::vector<bool>>& g) {
117.         this->g = g;
118.     }
119.
120.     void creaPolinomio() {
121.         sPolinomio = cromático(g);
122.     }
123.
124.     void traducePolinomio() {
125.         fun = traductor(g.size(), sPolinomio);
126.     }
127.
128.     __int128_t P(int k) {
129.         return evaluación(fun, k);
130.     }
131. };
132.

```

Adaptación del programa implementado en el servicio social del estudiante, se implementó como un módulo llamado “polinomio.h” [8].

12.7. Códigos fuentes implementados para los resultados de la sección 7

```

1. #include <fstream>
2. #include <iostream>
3. #include "montecarlo.h"
4. #include "ops128.h"
5. #include "polinomio.h"
6. #include <stdfloat>
7. #include <time.h>
8. #include <vector>
9.
10. double tp, tm;
11. std::ifstream ent;
12. std::ofstream sal;
13.
14. __int128_t res_polinomio(std::vector<std::vector<bool>>& g, int k) {
15.     polinomio p(g);
16.     clock_t t0 = clock();
17.     p.creaPolinomio();
18.     p.traducePolinomio();
19.     __int128_t res = p.P(k);
20.     clock_t t1 = clock();
21.     sal << "Usando el algoritmo de borrado contraccion:\n"
22.         << "P(G, " << k << ") = " << i128to_string(res) << "\n"
23.         << "Tiempo: " << double(t1 - t0) / CLOCKS_PER_SEC << "\n";
24.     tp += double(t1 - t0) / CLOCKS_PER_SEC;
25.     return res;
26. }
27.
28. monteCarlo::resultado res_montecarlo(std::vector<std::vector<bool>>& g, int k) {
29.     std::float128_t e = 0.01, z = 2.575;
30.     clock_t t0 = clock();
31.     monteCarlo mc(z, e, g, k);
32.     monteCarlo::resultado res = mc.intervalo();
33.     clock_t t1 = clock();

```

```

34.     sal << "Usando el metodo de Montecarlo:\n"
35.         << "Intervalo de confianza: [" << f128to_string(res.intervalo.first) << ", "
36.         << f128to_string(res.intervalo.second) << "]\n"
37.         << "Media: " << f128to_string(res.media) << "\n"
38.         << "Tiempo: " << double(t1 - t0) / CLOCKS_PER_SEC << "\n";
39.     tm += double(t1 - t0) / CLOCKS_PER_SEC;
40.     return res;
41. }
42.
43. void guiones() {
44.     for(int i = 0; i < 20; ++i) {
45.         sal << "-";
46.     }
47.     sal << "\n";
48. }
49.
50. void leerMatriz(std::vector<std::vector<bool>>& g) {
51.     for(int i = 0; i < g.size(); ++i) {
52.         for(int j = 0; j < g.size(); ++j) {
53.             int p;
54.             ent >> p;
55.             g[i][j] = p;
56.         }
57.     }
58. }
59.
60. void imprimeMatriz(std::vector<std::vector<bool>>& g) {
61.     for(int i = 0; i < g.size(); ++i) {
62.         for(int j = 0; j < g.size(); ++j) {
63.             sal << "[" << g[i][j] << " ";
64.         }
65.         sal << "\n";
66.     }
67. }
68.
69. bool compara(const __int128_t &poli, const monteCarlo::resultado &monte) {
70.     std::float128_t err = erel(poli, monte.media);
71.     sal << "Error relativo: " << f128to_string(err) << "\n";
72.     if(std::float128_t(poli) >= monte.intervalo.first && std::float128_t(poli) <=
monte.intervalo.second) {
73.         sal << "El resultado SI esta dentro del intervalo :)\n";
74.         return 1;
75.     } else {
76.         sal << "El resultado NO esta dentro del intervalo :)\n";
77.         return 0;
78.     }
79. }
80.
81. int main() {
82.     for(int u = 1; u <= 12; ++u) {
83.         tp = 0, tm = 0;
84.         clock_t t0 = clock();
85.         std::string sent = "n" + std::to_string(u) + "/ent.txt";
86.         ent = std::ifstream(sent);
87.         std::string ssal = "n" + std::to_string(u) + "/sal.txt";
88.         sal = std::ofstream(ssal);
89.         int m;
90.         ent >> m;
91.         int veces = 0;
92.         for(int i = 0; i < m; ++i) {
93.             int n = u;
94.             std::vector<std::vector<bool>> g(n, std::vector<bool>(n));
95.             leerMatriz(g);
96.             guiones();
97.             imprimeMatriz(g);

```

```

98.         guiones();
99.         for(int k = 0; k < 5; ++k) {
100.             __int128_t resp = res_polinomio(g, n + 5 * k);
101.             monteCarlo::resultado resm = res_montecarlo(g, n + 5 * k);
102.             veces += compara(resp, resm);
103.             guiones();
104.         }
105.     }
106.     guiones();
107.     sal << "Numero de vertices: " << u << "\n";
108.     sal << "Total de graficas: " << m << "\n";
109.     sal << "Total de evaluaciones calculadas: " << m * 5 << "\n";
110.     sal << "Intervalos de evaluaciones correctamente resueltos: " << veces << "\n";
111.     sal << "Tiempo medio Montecarlo: " << tm / (m * 5) << "\n";
112.     sal << "Tiempo medio Borrado-Contraccion: " << tp / (m * 5) << "\n";
113.     clock_t t1 = clock();
114.     std::cout << "n = " << u << " " << double(t1 - t0) / CLOCKS_PER_SEC << "s :)\n";
115. }
116. }
117.

```

Código para comparar el método de Montecarlo vs el algoritmo de borrado-contracción de la sección 7.1, los archivos donde se encuentran las gráficas están en la carpeta del proyecto completo.

```

1. #include <immintrin.h>
2. #include <iostream>
3. #include <random>
4.
5. struct random_rand {
6.     using result_type = uint64_t;
7.     uint64_t operator()() {
8.         uint64_t res;
9.         while (_rdrand64_step(&res) == 0) {
10.             continue;
11.         }
12.         return res;
13.     }
14.     static constexpr result_type min() {
15.         return std::numeric_limits<result_type>::min( );
16.     }
17.     static constexpr result_type max() {
18.         return std::numeric_limits<result_type>::max( );
19.     }
20. };
21.
22. void creamatriz(int n) {
23.     int mat[n][n] = {};
24.     random_rand gen;
25.     std::uniform_int_distribution<int> distribución(0, 1);
26.     for(int i = 0; i < n; ++i) {
27.         for(int j = i + 1; j < n; ++j) {
28.             mat[i][j] = distribución(gen);
29.             mat[j][i] = mat[i][j];
30.         }
31.     }
32.
33.     for(int i = 0; i < n; ++i) {
34.         for(int j = 0; j < n; ++j) {
35.             std::cout << mat[i][j] << " ";
36.         }
37.         std::cout << "\n";
38.     }
39.     std::cout << "\n";

```

```

40. }
41.
42. int main() {
43.     int n = 12;
44.     std::cout << 10 << "\n\n";
45.     for(int i = 0; i < 10; ++i) {
46.         creamatriz(n);
47.     }
48. }
49.

```

Código para generar gráficas con aristas aleatorias, para los casos de $9 \leq n \leq 12$ de la sección 7.1.

```

1. #include <iostream>
2.
3. void creaCompleta(int n) {
4.     for(int i = 0; i < n; ++i) {
5.         for(int j = 0; j < n; ++j) {
6.             std::cout << ((i != j) ? '1' : '0') << " ";
7.         }
8.         std::cout << "\n";
9.     }
10.    std::cout << "\n";
11. }
12.
13. int main() {
14.     int n;
15.     std::cin >> n;
16.     std::cout << n << "\n\n";
17.     for(int i = 1; i <= n; ++i) {
18.         creaCompleta(i);
19.     }
20. }
21.

```

Código para generar de gráficas completas de tamaño n .

```

1. #include "montecarlo.h"
2. #include <chrono>
3. #include <iostream>
4.
5. double tm;
6.
7. __int128_t P(int n, int k) {
8.     __int128_t res = 1;
9.     for(int i = 0; i < n; ++i) {
10.        res *= __int128_t(k - i);
11.    }
12.    return res;
13. }
14.
15. __int128_t res_polinomio(std::vector<std::vector<bool>>& g, int k) {
16.    __int128_t res = P(g.size(), k);
17.    clock_t t1 = clock();
18.    std::cout << "P(G, " << k << ") = " << i128to_string(res) << "\n";
19.    return res;
20. }
21.
22. monteCarlo::resultado res_montecarlo(std::vector<std::vector<bool>>& g, int k) {
23.    std::float128_t e = 0.01, z = 2.575;
24.    const auto t0 = std::chrono::high_resolution_clock::now();
25.    monteCarlo mc(z, e, g, k);
26.    monteCarlo::resultado res = mc.intervalo();

```

```

27.     const auto t1 = std::chrono::high_resolution_clock::now();
28.     double tiempo = std::chrono::duration_cast<std::chrono::duration<double>>(t1 -
t0).count());
29.     std::cout << "Usando el metodo de Montecarlo:\n"
30.         << "Intervalo de confianza: [" << f128to_string(res.intervalo.first) << ", "
31.         << f128to_string(res.intervalo.second) << "]\n"
32.         << "Media: " << f128to_string(res.media) << "\n"
33.         << "Tiempo: " << tiempo << "\n";
34.     tm += tiempo;
35.     return res;
36. }
37.
38. void guiones() {
39.     for(int i = 0; i < 20; ++i) {
40.         std::cout << "-";
41.     }
42.     std::cout << "\n";
43. }
44.
45. void leerMatriz(std::vector<std::vector<bool>>& g) {
46.     for(int i = 0; i < g.size(); ++i) {
47.         for(int j = 0; j < g.size(); ++j) {
48.             int p;
49.             std::cin >> p;
50.             g[i][j] = p;
51.         }
52.     }
53. }
54.
55. bool compara(const __int128_t &poli, const monteCarlo::resultado &monte) {
56.     std::float128_t err = erel(poli, monte.media);
57.     std::cout << "Error relativo: " << f128to_string(err) << "\n";
58.     if(std::float128_t(poli) >= monte.intervalo.first && std::float128_t(poli) <=
monte.intervalo.second) {
59.         std::cout << "El resultado SI esta dentro del intervalo :)\n";
60.         return 1;
61.     } else {
62.         std::cout << "El resultado NO esta dentro del intervalo :)\n";
63.         return 0;
64.     }
65. }
66.
67. void imprimeMatriz(std::vector<std::vector<bool>>& g) {
68.     for(int i = 0; i < g.size(); ++i) {
69.         for(int j = 0; j < g.size(); ++j) {
70.             std::cout << "[" << g[i][j] << "]";
71.         }
72.         std::cout << "\n";
73.     }
74. }
75.
76. int main() {
77.     int n;
78.     std::cin >> n;
79.     int veces = 0;
80.     std::vector<double> tiempos(n);
81.     for(int i = 1; i <= n; ++i) {
82.         tm = 0;
83.         std::vector<std::vector<bool>> g(i, std::vector<bool>(i));
84.         leerMatriz(g);
85.         guiones();
86.         imprimeMatriz(g);
87.         guiones();
88.         for(int j = 1; j <= 5; ++j) {
89.             __int128_t resp = res_polinomio(g, 2 * i + 5 * j);

```

```

90.         monteCarlo::resultado resm = res_montecarlo(g, 2 * i + 5 * j);
91.         veces += compara(resp, resm);
92.         guiones();
93.     }
94.     tiempos[i - 1] = tm / 5.0;
95. }
96. std::cout << "Evaluaciones: " << n * 5 << "\n"
97.     << "Montecarlos correctos: " << veces << "\n";
98. guiones();
99. std::cout << "Vértices\tTiempo\n";
100. for(int i = 0; i < n; ++i) {
101.     std::cout << (i + 1) << "\t" << tiempos[i] << "\n";
102. }
103. return 0;
104. }
105.

```

Código para comparar el método de Montecarlo para las gráficas de completas de la sección 7.2.1 y 7.2.4.

```

1. #include <iostream>
2. #include <vector>
3.
4. void creaGráfica(int n) {
5.     for(int i = 0; i < n; ++i) {
6.         for(int j = 0; j < n; ++j) {
7.             std::cout << "0 ";
8.         }
9.         std::cout << "\n";
10.    }
11.    std::cout << "\n";
12. }
13.
14. int main() {
15.     int n;
16.     std::cin >> n;
17.     std::cout << n << "\n\n";
18.     for(int i = 0; i < n; ++i) {
19.         creaGráfica(i + 1);
20.     }
21.     return 0;
22. }
23.

```

Código para generar gráficas sin aristas de tamaño n .

```

1. #include "montecarlo.h"
2. #include <chrono>
3. #include <iostream>
4.
5. double tm;
6.
7. __int128_t P(int n, int k) {
8.     return potencia(k, n);
9. }
10.
11. __int128_t res_polinomio(std::vector<std::vector<bool>>& g, int k) {
12.     return P(g.size(), k);
13. }
14.
15. monteCarlo::resultado res_montecarlo(const std::vector<std::vector<bool>>& g, int k) {
16.     std::float128_t e = 0.01, z = 2.575;
17.     monteCarlo mc(z, e, g, k);
18.     const auto t0 = std::chrono::high_resolution_clock::now();

```



```

19.     monteCarlo::resultado res = mc.intervalo();
20.     const auto t1 = std::chrono::high_resolution_clock::now();
21.     tm = std::chrono::duration_cast<std::chrono::duration<double>>(t1 - t0).count();
22.     return res;
23. }
24.
25. void leerMatriz(std::vector<std::vector<bool>>& g) {
26.     for(int i = 0; i < g.size(); ++i) {
27.         for(int j = 0; j < g.size(); ++j) {
28.             int p;
29.             std::cin >> p;
30.             g[i][j] = p;
31.         }
32.     }
33. }
34.
35. bool compara(const __int128_t &poli, const monteCarlo::resultado &monte) {
36.     std::float128_t err = erel(poli, monte.media);
37.     if(std::float128_t(poli) >= monte.intervalo.first && std::float128_t(poli) <=
monte.intervalo.second) {
38.         return 1;
39.     } else {
40.         return 0;
41.     }
42. }
43.
44. int main() {
45.     int n;
46.     std::cin >> n;
47.     int veces = 0;
48.     std::cout << "Vértices\tTiempo\n";
49.     for(int i = 1; i <= n; ++i) {
50.         std::vector<std::vector<bool>> g(i, std::vector<bool>(i));
51.         leerMatriz(g);
52.         __int128_t resp = res_polinomio(g, 2);
53.         monteCarlo::resultado resm = res_montecarlo(g, 2);
54.         veces += compara(resp, resm);
55.         std::cout << i << "\t" << tm << "\n";
56.     }
57.     std::cout << "Evaluaciones: " << n << "\n"
58.         << "Montecarlos correctos: " << veces << "\n";
59.     return 0;
60. }
61.

```

Código para comparar el método de Montecarlo para las gráficas sin aristas de la sección 7.2.2.

```

1. #include <iostream>
2.
3. void creaCamino(int n) {
4.     for(int i = 0; i < n; ++i) {
5.         for(int j = 0; j < n; ++j) {
6.             std::cout << ((j == (i - 1) || j == (i + 1)) ? '1' : '0') << " ";
7.         }
8.         std::cout << "\n";
9.     }
10.    std::cout << "\n";
11. }
12.
13. int main() {
14.     int n;
15.     std::cin >> n;
16.     std::cout << n << "\n\n";
17.     for(int i = 1; i <= n; ++i) {

```

```

18.         creaCamino(i);
19.     }
20. }
21.

```

Código para generar caminos de tamaño n .

```

1. #include "montecarlo.h"
2. #include <chrono>
3. #include <iostream>
4.
5. double tm;
6.
7. __int128_t P(int n, int k) {
8.     __int128_t res = k;
9.     for(int i = 1; i < n; ++i) {
10.         res *= __int128_t(k - 1);
11.     }
12.     return res;
13. }
14.
15. __int128_t res_polinomio(std::vector<std::vector<bool>>& g, int k) {
16.     __int128_t res = P(g.size(), k);
17.     clock_t t1 = clock();
18.     std::cout << "P(G, " << k << ") = " << i128to_string(res) << "\n";
19.     return res;
20. }
21.
22. monteCarlo::resultado res_montecarlo(std::vector<std::vector<bool>>& g, int k) {
23.     std::float128_t e = 0.01, z = 2.575;
24.     const auto t0 = std::chrono::high_resolution_clock::now();
25.     monteCarlo mc(z, e, g, k);
26.     monteCarlo::resultado res = mc.intervalo();
27.     const auto t1 = std::chrono::high_resolution_clock::now();
28.     double tiempo = std::chrono::duration_cast<std::chrono::duration<double>>(t1 -
t0).count());
29.     std::cout << "Usando el metodo de Montecarlo:\n"
30.         << "Intervalo de confianza: [" << f128to_string(res.intervalo.first) << ", "
31.         << f128to_string(res.intervalo.second) << "]\n"
32.         << "Media: " << f128to_string(res.media) << "\n"
33.         << "Tiempo: " << tiempo << "\n";
34.     tm += tiempo;
35.     return res;
36. }
37.
38. void guiones() {
39.     for(int i = 0; i < 20; ++i) {
40.         std::cout << "-";
41.     }
42.     std::cout << "\n";
43. }
44.
45. void leerMatriz(std::vector<std::vector<bool>>& g) {
46.     for(int i = 0; i < g.size(); ++i) {
47.         for(int j = 0; j < g.size(); ++j) {
48.             int p;
49.             std::cin >> p;
50.             g[i][j] = p;
51.         }
52.     }
53. }
54.
55. bool compara(const __int128_t &poli, const monteCarlo::resultado &monte) {
56.     std::float128_t err = erel(poli, monte.media);

```

```

57.     std::cout << "Error relativo: " << f128to_string(err) << "\n";
58.     if(std::float128_t(poli) >= monte.intervalo.first && std::float128_t(poli) <=
monte.intervalo.second) {
59.         std::cout << "El resultado SI esta dentro del intervalo :)\n";
60.         return 1;
61.     } else {
62.         std::cout << "El resultado NO esta dentro del intervalo :)\n";
63.         return 0;
64.     }
65. }
66.
67. void imprimeMatriz(std::vector<std::vector<bool>>& g) {
68.     for(int i = 0; i < g.size(); ++i) {
69.         for(int j = 0; j < g.size(); ++j) {
70.             std::cout << "[" << g[i][j] << "]";
71.         }
72.         std::cout << "\n";
73.     }
74. }
75.
76. int main() {
77.     int n;
78.     std::cin >> n;
79.     int veces = 0;
80.     std::vector<double> tiempos(n);
81.     for(int i = 1; i <= n; ++i) {
82.         tm = 0;
83.         std::vector<std::vector<bool>> g(i, std::vector<bool>(i));
84.         leerMatriz(g);
85.         guiones();
86.         imprimeMatriz(g);
87.         guiones();
88.         for(int j = 0; j < 5; ++j) {
89.             __int128_t resp = res_polinomio(g, 5 + j * 2);
90.             monteCarlo::resultado resm = res_montecarlo(g, 5 + j * 2);
91.             veces += compara(resp, resm);
92.             guiones();
93.         }
94.         tiempos[i - 1] = tm / 5.0;
95.     }
96.     std::cout << "Evaluaciones: " << n * 5 << "\n"
97.         << "Montecarlos correctos: " << veces << "\n";
98.     guiones();
99.     std::cout << "Vértices\tTiempo\n";
100.    for(int i = 0; i < n; ++i) {
101.        std::cout << (i + 1) << "\t" << tiempos[i] << "\n";
102.    }
103.    return 0;
104. }
105.

```

Código para comparar el método de Montecarlo para las gráficas de caminos de la sección 7.2.3.